

Get Started with Intel® Advisor

Contents

Chapter 1: Get Started with Intel® Advisor

Before You Begin	4
Discover Where Vectorization Pays Off The Most	7
Identify Performance Bottlenecks Using CPU Roofline	16
Identify High-impact Opportunities to Offload to GPU	18
Measure GPU Performance Using GPU Roofline	26
Prototype Threading Designs.....	29
Next Steps	38
Learn More	38
Notices and Disclaimers.....	40

Get Started with Intel® Advisor

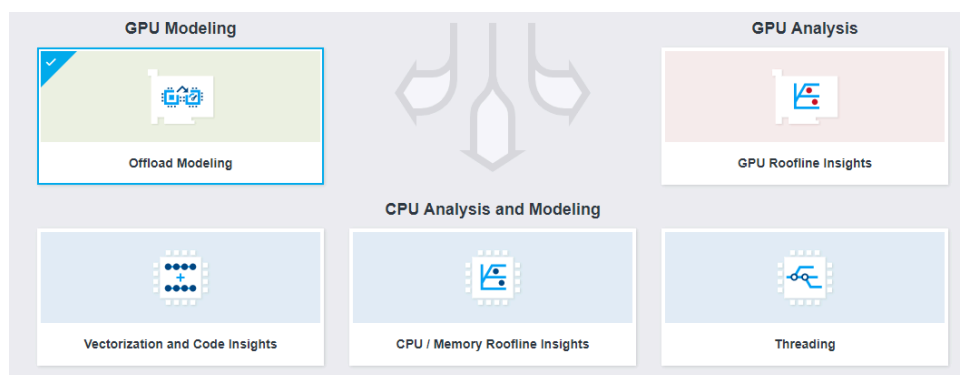
Use **Intel® Advisor** to make sure your C, C++, Fortran, OpenMP*, SYCL, Python*, or OpenCL™ application realizes full performance potential on modern processors. You can download the Intel Advisor as:

- [Standalone Intel Advisor](#)
- Part of [Intel® oneAPI Base Toolkit](#)

After you download and install the Intel Advisor, start with [Before You Begin](#) instructions to launch the tool and set up the environment. Continue to run Intel Advisor perspectives to analyze your code.

In the Intel Advisor, a *perspective* is a predefined analysis workflow enabled by multiple profiling steps and options and represented as a specific data layout optimized to solve a problem. Depending on your analysis goal, you can follow one or more of the perspectives to understand application performance and identify bottlenecks.

This document summarizes typical Intel Advisor workflows to optimize application performance. You can analyze your code from the following perspectives:



Vectorization and Code Insights

Find unvectorized and under-vectorized loops and functions in your application and get code-specific recommendations for how improving application performance with vectorization. This includes applying single instruction multiple data (SIMD) instructions, such as Intel® Advanced Vector Extensions and Intel® Advanced Vector Extensions 512, on multiple data objects to run in parallel within a single CPU core. You can analyze a C, C++, Fortran, OpenMP, or mixed Python application.

See [Discover Where Vectorization Pays Off The Most](#) for quick-start workflow with a sample application.

CPU/Memory Roofline Insights

Visualize your application performance against CPU limitations as a Roofline chart to see how well your application uses hardware resources, what is the main limiting factor, and what are the optimization candidates. You can analyze a C, C++, Fortran, OpenMP, or mixed Python application.

See [Identify Performance Bottlenecks Using CPU Roofline](#) for quick-start workflow with a sample application.

Offload Modeling

For an application running on a CPU, determine if your application can benefit from porting to a GPU, identify the most profitable offload opportunities and potential bottlenecks. You can analyze a C, C++, Fortran, SYCL, OpenCL application, or an application with OpenMP target pragmas.

For an application running on a GPU, determine if your application can get speedup from running on a different next-generation GPU and identify potential bottlenecks. You can analyze a SYCL, OpenCL application, or an application with OpenMP target pragmas.

See [Identify High-impact Opportunities to Offload to GPU](#) for quick-start CPU-to-GPU workflow with a sample application.

GPU Roofline Insights

Profile a GPU application and visualize kernel performance against GPU limitations as a Roofline chart to identify the main bottlenecks and limiting factor, get recommendations to optimize performance on GPU. You can analyze a SYCL, OpenCL application, or an application with OpenMP target pragmas.

See [Measure GPU Performance Using GPU Roofline](#) for quick-start workflow with a sample application.

Threading

Prototype different threading design options and project scaling on systems with larger core counts for your application without disrupting development and find performance issues to fix them before implementing parallelism. You can analyze a C, C++, or Fortran application.

See [Prototype Threading Designs](#) for quick-start workflow with a sample application.

Before You Begin

Download and install Intel® Advisor:

- [Standalone installation](#)
- Part of [Intel® oneAPI Base Toolkit](#)

Linux* OS

The following sections describe how to start working with Intel Advisor on Linux OS.

Default Installation Path

By default, the <install-dir> for all oneAPI components is as follows:

- /opt/intel/oneapi/ for root users
- \$HOME/intel/oneapi/ for non-root users

Set Up Environment

If you plan to use Intel Advisor command line interface (CLI) or launch the graphical user interface (GUI) from the command line, you need to set up the environment variables first.

To set up the environment for Intel Advisor, run the `setvars.sh` script from a terminal as follows.

```
source $HOME/intel/oneapi/setvars.sh
```

If you installed Intel Advisor as a root user or to a different location, make sure to replace the path to the script appropriately.

Launch Intel Advisor

You can launch Intel Advisor with one of the following methods:

Intel Advisor GUI

Launch the Intel Advisor GUI from the terminal as follows:

```
advisor-gui
```

Intel Advisor CLI

Run the `advisor` command with appropriate actions and action options from the terminal. The general command syntax is:

```
advisor <--action> [--<action-options>] [--<global-options>] [--] <target> [<target options>]
```

For example, analyze a CPU application `myApplication` with the Survey analysis, which is the basic analysis for all Intel Advisor perspectives:

```
advisor --collect=survey --project-dir=./advi_results -- ./myApplication
```

Windows* OS

The following sections describe how to start working with Intel Advisor on Windows OS.

Default Installation Path

By default, the `<install-dir>` for all oneAPI components is `C:\Program Files (x86)\Intel\oneAPI\` (on certain systems, instead of `Program Files (x86)`, the directory name is `Program Files`).

Set Up Environment

If you plan to use Intel Advisor command line interface (CLI) or launch the graphical user interface (GUI) from the command line, you need to set up the environment variables first.

To set up the environment for Intel Advisor, run the `setvars.bat` script from a command prompt as follows.

```
"C:\Program Files (x86)\Intel\oneAPI\setvars.bat"
```

If you installed Intel Advisor as to a different location, make sure to replace the path to the script appropriately.

Launch Intel Advisor

You can launch Intel Advisor with one of the following methods:

Intel Advisor GUI

To launch the Intel Advisor GUI:

- Run the Intel Advisor from the **Start** menu.
- Run the Intel Advisor from the command prompt:

```
advisor-gui
```

Intel Advisor Microsoft Visual Studio* Integration

1. Open a solution in the Microsoft Visual Studio.
2. Click the  icon on the toolbar or go to **Tools > Intel Advisor > Open Perspective Selector**.

If you do not see the icon, right-click the toolbar and select **Intel Advisor** from the context menu.

Intel Advisor CLI

Run the `advisor` command with appropriate actions and action options from the command prompt. The general command syntax is:

```
advisor <--action> [--<action-options>] [--<global-options>] [--] <target> [<target options>]
```

For example, analyze a CPU application `myApplication` with the Survey analysis, which is the basic analysis for all Intel Advisor perspectives:

```
advisor --collect=survey --project-dir=.\\advi_results -- myApplication.exe
```

macOS*

The following sections describe how to start working with Intel Advisor on macOS.

Explore a step-by-step instruction how to view your Intel Advisor results on a macOS* machine in the [Intel Advisor Cookbook: Analyze Performance Remotely and Visualize Results on a Local macOS* System](#).

Default Installation Path

By default, the `<install-dir>` for all oneAPI components is `/opt/intel/oneapi/`.

Set Up Environment

If you plan to run the Intel® Advisor graphical user interface (GUI) from the command line, you need to set up the environment variables first.

To set up the environment for Intel Advisor, run the `setvars` script from a terminal as one of the following.

```
/opt/intel/oneapi/setvars.sh
```

```
/opt/intel/oneapi/setvars.csh
```

If you installed Intel Advisor as a root user or to a different location, make sure to replace the path to the script appropriately.

Launch Intel Advisor

On macOS, you cannot profile your application with Intel Advisor, but you can view the results collected on a different OS using the GUI. You can launch Intel Advisor GUI to view the collected results with one of the following methods:

- Launch the Intel Advisor GUI, go to **File > Open > Project...**, and navigate to the result to open.
- Launch the Intel Advisor GUI from the terminal as follows:

```
advisor-gui <result-path>
```

where `<result-path>` is the path to a result project or a result snapshot.

Create a Project

Create a Project

A *project* is a reusable container that includes target executable location, sources, and binaries, collection properties, and collection results.

Prerequisite: Before creating a project, build your application in release mode. For more information on building applications, see [Build Target Application](#).

To create a project in the standalone GUI:

1. Choose **File > New > Project...** to open the **Create a Project** dialog box.
2. Specify a name and location for your project.
3. Click the **Create Project** button to open the **Project Properties** dialog box.
4. In the **Analysis Target** tab, click **Browse** near the **Application** field and select the application executable file.
5. Optional: In the **Binary/Symbol Search** and **Source Search** tabs, specify search directories.
6. Click **OK** to create the project.

If you work in the Microsoft Visual Studio*, a project is created automatically for the built application when you open the Intel® Advisor.

If you work with the `advisor` command line interface, a project is created automatically when you run the first analysis for the specified application.

Discover Where Vectorization Pays Off The Most

With the Vectorization and Code Insights perspective, you can identify loops and unctions in your application that can benefit most from vector parallelism, locate un-vectorized and under-vectorized time-consuming functions/loops and calculate estimated performance gain achieved by vectorization.

This page explains how to profile the `vec_samples` application and identify vectorization hotspots to improve performance of your code. You can also use your own application to follow the instructions below.

The screenshot displays the Intel Advisor interface for the `vec_samples` application. The top navigation bar includes tabs for Summary, Survey & Roofline, and Refinement Reports. The main section is titled "Vectorization and Code Insights" and provides a brief description of the perspective's purpose.

Program Metrics

Program Elapsed Time: 4.05s
 Vector Instruction Set: SSE2, SSE
 Number of CPU Threads: 1

Performance Characteristics

Metrics	Total	Percentage
CPU Time	3.95s	100%
Time in 2 Vectorized Loops	1.44s	36.6%
Time in scalar code	2.50s	63.4%

Vectorization Gain/Efficiency

Vectorized Loops Gain/Efficiency	2.39x	60%
Program Approximate Gain	1.51x	

Per Program Recommendations

Higher instruction set architecture (ISA) available
 Consider recompiling your application using a higher ISA. [Show more](#)

Top Time-Consuming Loops

Loop	Self Time	Total Time	Vector Efficiency
loop in <code>matvec</code> at <code>Multiply.c:82</code>	2.023s	2.023s	~25%
loop in <code>matvec</code> at <code>Multiply.c:69</code>	0.765s	0.765s	~40%
loop in <code>matvec</code> at <code>Multiply.c:60</code>	0.679s	0.679s	~40%
loop in <code>matvec</code> at <code>Multiply.c:49</code>	0.451s	3.919s	
loop in <code>main</code> at <code>Driver.c:155</code>	<0.001s	3.935s	

Refinement Analysis Data

Recommendations

- Force the compiler to align loop code loop in `matvec` at `Multiply.c:82`
- Target the AVX2 ISA loop in `matvec` at `Multiply.c:82`
- Target the AVX2 ISA loop in `matvec` at `Multiply.c:69`
- Force the compiler to align loop code loop in `matvec` at `Multiply.c:69`
- Specify the expected loop trip count loop in `matvec` at `Multiply.c:60`

Follow the steps:

1. Unpack and build your application.
2. Establish performance baseline.

3. [Disambiguate pointers.](#)
4. [Generate instructions for the highest instruction set architecture.](#)
5. [Next steps](#)

Prerequisites

1. Install the Intel Advisor as a [standalone](#) or as [part of Intel® oneAPI Base Toolkit](#). For installation instructions, see [Install Intel Advisor](#) in the user guide.
2. Install the Intel® C++ Compiler Classic as a [standalone](#) or as [part of Intel® oneAPI HPC Toolkit](#). For installation instructions, see [Intel® oneAPI Toolkits Installation Guide](#).
3. Set up environment variables for the Intel Advisor and Intel® C++ Compiler Classic. For example, run the `setvars` script in the installation directory.

This document assumes you installed the tools to a [default location](#). If you installed the tools to a different location, make sure to replace the default path in the commands below.

Important Do not close the terminal or command prompt after setting the environment variables. Otherwise, the environment resets.

Unpack and Build Your Application

On Linux* OS

From the terminal where you set the environment variables:

1. Go to `/opt/intel/oneapi/advisor/latest/samples/en/C++` directory.
2. Copy the `vec_samples.tgz` file to a writable directory or share on your system.
3. Extract the sample from the `.tgz` archive.
4. Change directory to the `vec_samples/` directory in its unzipped location.
5. Build the sample application in release mode:

```
make baseline
```

The command build the application with the `-O2 -g` compiler options. For details about building your own applications, see [Build Target Application](#).

6. Run the application to verify the build:

```
make baseline
```

You should see an output similar to the following indicating that you successfully built the application:

```
ROW:47 COL: 47
Execution time is 6.020 seconds
GigaFlops = 0.733887
Sum of result = 254364.540283
```

On Windows* OS

From the command prompt where you set the environment variables:

1. Go to `C:\Program Files (x86)\Intel\oneAPI\advisor\latest\samples\en\C++` directory.
2. Copy the `vec_samples.zip` file to a writable directory or share on your system.
3. Extract the sample from the `.zip` archive.
4. Change directory to the `vec_samples\` directory in its unzipped location.
5. Build the sample application in release mode as follows:

```
build.bat baseline
```

The script builds the application with the `/O2 /Qstd=c99 /fp:fast /Isrc /Zi /Qopenmp` compiler options. For details about building your own applications, see [Build Target Application](#).

6. Run the sample application to verify the build:

```
vec_samples.exe
```

You should see an output similar to the following indicating that you successfully built the application:

```
ROW:47 COL: 47
Execution time is 6.020 seconds
GigaFlops = 0.733887
Sum of result = 254364.540283
```

Establish Performance Baseline

Run Vectorization and Code Insights from Graphical User Interface (GUI)

1. From the terminal or command prompt where you set the environment variables, launch the Intel Advisor GUI:

```
advisor-gui
```

2. Create a project for the just-built `vec_samples` application. For details, see [Before You Begin](#).

When in the **Project Properties** dialog box, make sure the **Inherit settings from Survey Hotspots Analysis Type** checkbox is selected in the **Trip Counts and FLOP Analysis, Dependencies Analysis**, and **Memory Access Patterns Analysis** types.

3. In the **Perspective Selector** window, choose the Vectorization and Code Insights perspective.
4. In the **Analysis Workflow** pane, set data collection accuracy level to **Low**, and click the



button to run the perspective.

At this accuracy level, Intel Advisor runs Survey analysis and collects performance metrics of your application to locate under- and non-vectorized hotspots.

Run Vectorization and Code Insights from Command Line Interface (CLI)

On Linux OS

From the command prompt where you set the environment variables:

1. Collect Survey data using the following command:

```
advisor --collect=survey --project-dir=./results -- ./vec_samples
```

2. Generate a Survey report using the following command:

```
advisor --report=survey --project-dir=./results
```

The report summary will be printed to the terminal or command prompt. A copy of this report is saved into `./vec_samples/e000/hs000/advisor-survey.txt`.

When the analysis execution completes, the `vec_samples` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

On Windows OS

From the command prompt where you set the environment variables:

1. Collect Survey data using the following command:

```
advisor --collect=survey --project-dir=./results -- vec_samples.exe
```

2. Generate a Survey report using the following command:

```
advisor --report=survey --project-dir=./results
```

The report will be printed to the terminal or command prompt. A copy of this report is saved into `./vec_samples/e000/hs000/advisor-survey.txt`.

When the analysis execution completes, the `vec_samples` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

Examine Results

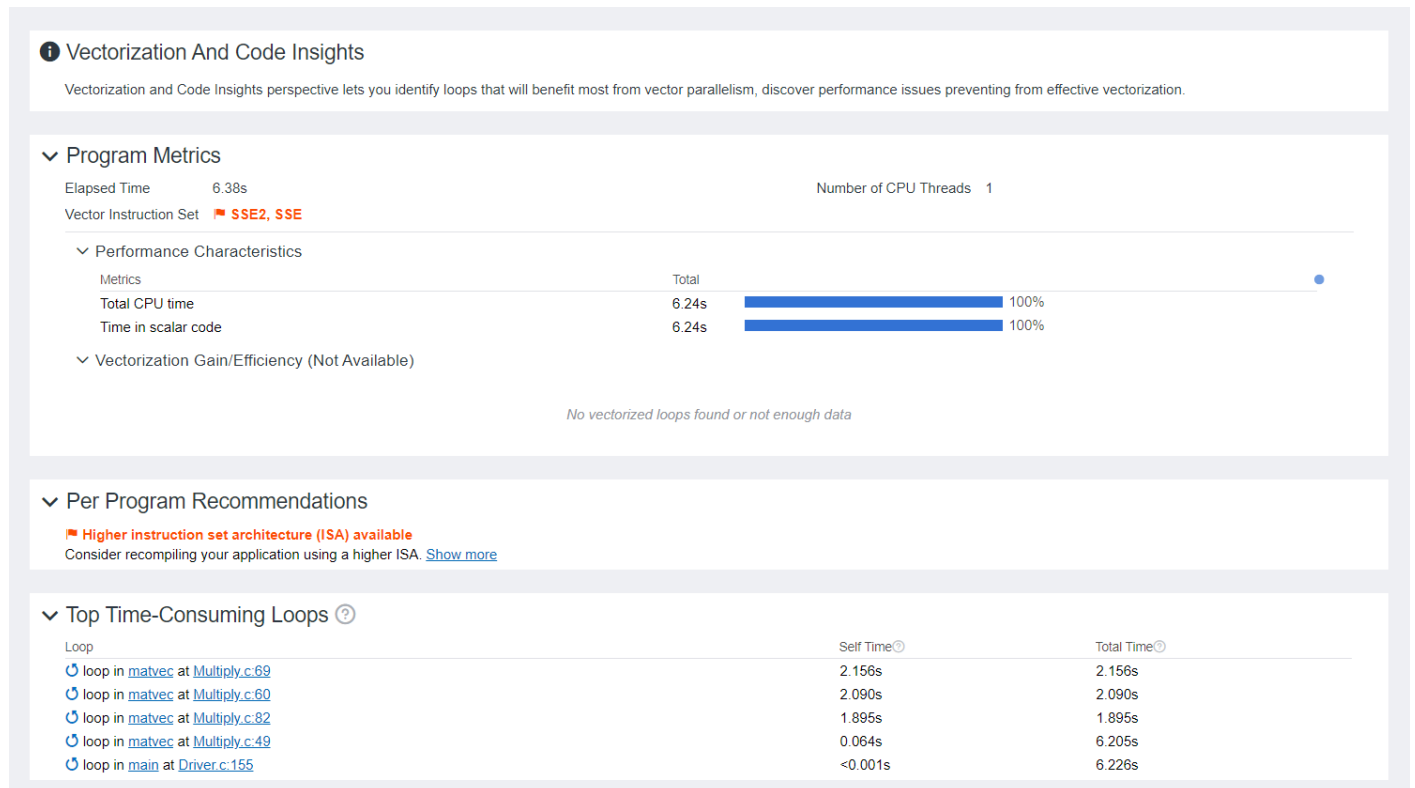
If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

If you collect data using CLI, open the results in GUI using the following command:

```
advisor-gui ./results
```

If the result does not open automatically, click **Show Result**.

When you open the Vectorization and Code Insights result in GUI, Intel Advisor shows the **Summary** tab first. This window is a dashboard containing the main information about application execution, performance hints, and indication of vectorization problems in your application.



In the **Summary** window, notice the following:

1. Assess your application performance using the **Elapsed Time** metric in the **Program Metrics** pane. Each improvement you make to under- and unvectorized functions/loops contributes to improvement of this metric. Consider revising program elapsed time after every iteration of running the perspective.
2. In the **Program Metrics** pane, **Time in scalar code** is 100% and the **Vectorization Gain/Efficiency** is empty. It means there are no vectorized loops in the application.
3. In the **Program Metrics** pane, **Vector Instruction Set** is **SSE2** and **SSE**. This metric is highlighted in red. Hover over the metric value to see a warning that a higher instruction set architecture is available. This warning is also reported in the **Per Program Recommendations** pane. Consider generating instructions for it and recompiling your application to improve performance.
4. View the top hotspots for optimization in the **Top Time-Consuming Loops** pane. Click the largest hotspot to view detailed metrics for it in the **Survey Report**.

Switch to the **Survey & Roofline** tab, you can analyze performance for each loop/function in the application.

Higher instruction set architecture (ISA) available
Consider recompiling your application using a higher ISA.

Function Call Sites and Loops	Performance Issues	CPU Time		Type	Why No Vectorization?	Vectorized Loops			Instruction Set Analysis
		Total Time	Self Time			Vector...	Gain E...	VL (Ve...	
[loop in matvec at Multiply.c:69]	2 Assumed dep...	2.156s	2.156s	Scalar	vector dependence pre...				
[loop in matvec at Multiply.c:60]	2 Assumed depe...	2.090s	2.090s	Scalar	vector dependence preve...				
[loop in matvec at Multiply.c:82]	3 Assumed depe...	1.895s	1.895s	Scalar	vector dependence preve...				
[loop in matvec at Multiply.c:49]		6.205s	99.4%	Scalar	outer loop was not auto-...				
f matvec		6.226s	99.7%	Function					
f _srt_common_main_seh		6.242s	100.0%	Function					
[loop in main at Driver.c:155]	1 Data type conv...	6.226s	99.7%	Scalar	loop with function call n...				Type Conversions
f main	1 Data type conv...	6.242s	100.0%	Function					Shifts; Shuffles; Type Conve

1. The **Elapsed time** value in the top left corner. This is the baseline against which subsequent improvements will be measured.
2. In the **Type** column, all detected loops are *scalar*.
3. In the **Why No Vectorization?** column, the compiler detected or assumed a vector dependence in most loops.
4. For one of the loops where the compiler detected or assumed a vector dependence, click the  control to display *how-can-I-fix-this-issue?* information in the **Why No Vectorization?** pane.
5. Review the **Summary** window, which appears after the perspective executes. This window is a dashboard containing the main information about application execution, performance hints, and indication of vectorization problems in your application.

Create a Read-only Snapshot for the Baseline Result

Create a read-only result snapshot, which you can share or compare with other results. To do that:

1. Click the  icon.
2. Type `snapshot_baseline` in the **Result** name field.
3. Select the **Pack into archive** checkbox to enable the **Result path** field.
4. Browse to a desired location, then click the **OK** button to save a read-only snapshot of the current result.
5. If the **Survey Report** remains grayed out after the snapshot process is complete, click anywhere on the report.

To review performance improvements, open the saved result snapshots and compare the metrics with those in the `snapshot_baseline` snapshot.

Disambiguate Pointers

Two pointers are aliased if both point to the same memory location. Storing to memory using a pointer that might be aliased may prevent some optimizations. For example, it may create a dependency between loop iterations that would make vectorization unsafe. Sometimes the compiler can generate both a vectorized and a non-vectorized version of a loop and test for aliasing at runtime to select the appropriate code path. If you know pointers do not alias, and inform the compiler, it can avoid the runtime check and generate a single vectorized code path.

In `Multiply.c`, the compiler generates runtime checks to determine if point `b` in function `matvec(FTYPE a[][COLWIDTH], FTYPE b[], FTYPE x[])` is aliased to either `a` or `x`. If `Multiply.c` is compiled with the `NOALIAS` macro, the restrict qualifier of argument `b` informs the compiler the pointer does not alias with any other pointer and array `b` does not overlap with `a` or `x`.

To see if the `NOALIAS` macro improves performance, do the following:

On Linux OS

From the same terminal window:

1. Navigate to the `vec_samples/` directory.

2. Rebuild the target application with the `NOALIAS` macro:

```
make noalias
```

The command builds the application with the following compiler options: `-O2 -g -D NOALIAS`.

3. Rerun the Vectorization and Code Insights perspective from GUI or CLI with the same configuration as for the baseline result. See the sections above for instructions.

On Windows OS

From the same terminal window:

1. Navigate to the `vec_samples` directory.
2. Rebuild the target application with the `NOALIAS` macro:

```
build.bat noalias
```

The script builds the application with the following compiler options: `/O2 /Qstd=c99 /fp:fast /Isrc /Zi /Qopenmp /DNOALIAS`.

3. Rerun the Vectorization and Code Insights perspective from GUI or CLI with the same configuration as for the baseline result. See the sections above for instructions.

View the Results

If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

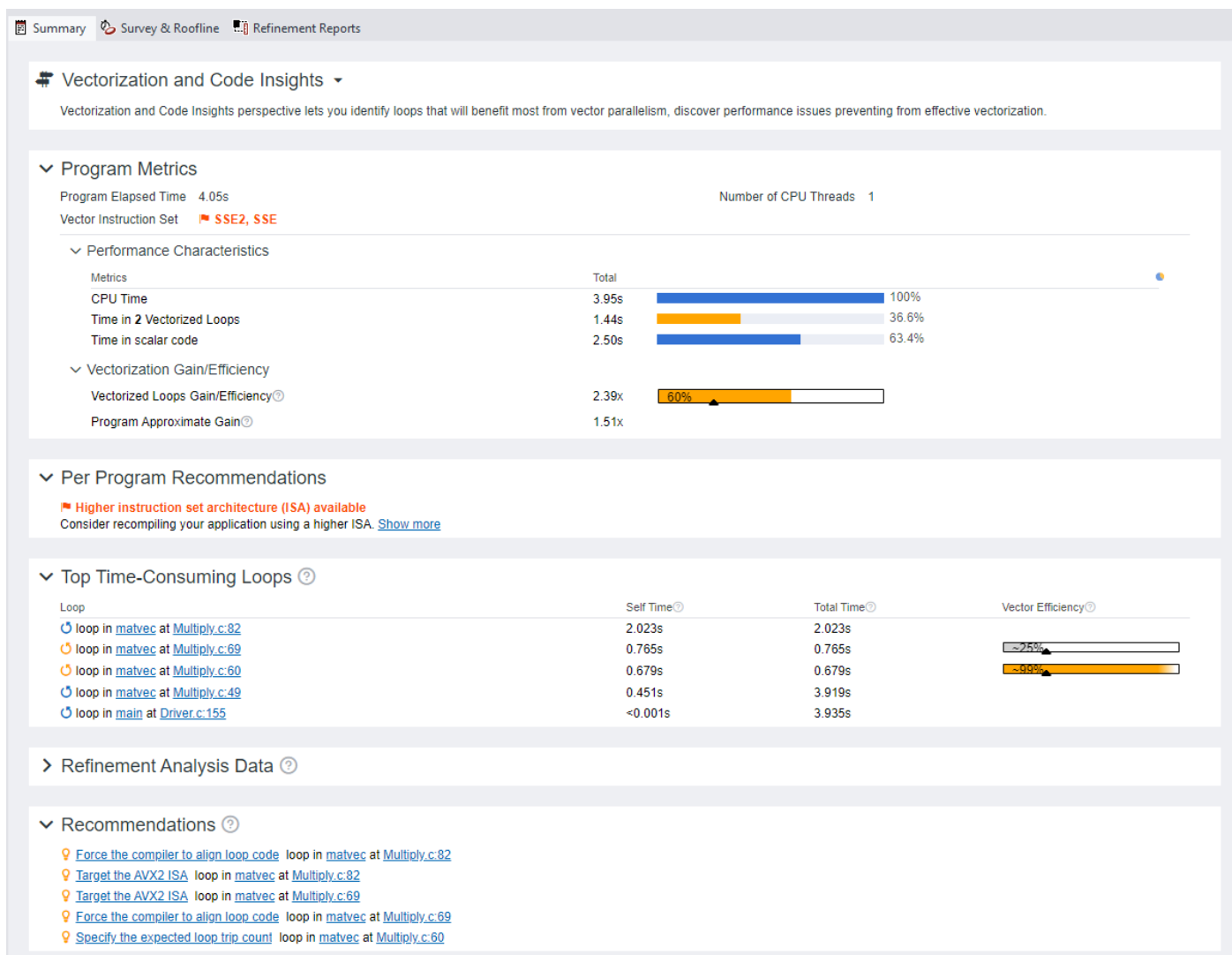
If you collect data using CLI, open the results in GUI using the following command:

```
advisor-gui ./vec_samples
```

If the result does not open automatically, click **Show Result**.

Check changes in the **Summary** window:

1. In the **Program Metrics** pane, a new metric **Time in 2 Vectorized Loops** appeared meaning that the compiler vectorized two loops. The time in the vectorized loops is 36.6% of the application execution time.
2. Examine the **Vectorization Gain/Efficiency** section of the pane. The loops are vectorized with 60% efficiency and have 2.39x speedup compared to their scalar version, but there is still room for more improvement. The whole application has 1.51x speedup compared to the fully scalar version.
3. The **Elapsed time** improves substantially.



Open the **Survey & Roofline** tab to assess the changes in application performance. In the report, notice the following:

1. The compiler successfully vectorizes two loops: in `matvec` at `Multiply.c:69` and in `matvec` at `Multiply.c:60`.

The loop in `matvec` at `Multiply.c:60` has a high efficiency (99%) and 3.96x estimated gain. The `matvec` at `Multiply.c:69` efficiency is lower (25%) and the bar is gray, which means that the achieved vectorization efficiency is lower than the original scalar loop efficiency. Hover over a bar in the Efficiency column to see the explanation for the estimated efficiency.

2. Click the  icon next to the two vectorized loops. Notice both loops have a remainder loop present. Click the  icon in the **Trips Counts** column set to expand it. The remainder loops are present because the trip count values for the remainder loops are not a multiple of the **VL (Vector Length)** value.

Function Call Sites and Loops	Performance Issues	CPU Time		Type	Why No Vectorization?	Vectorized Loops			Instruction Set Analysis	
		Total Time	Self Time			Vect...	Efficiency	Gain...	VL (...)	Traits
[loop in matvec at Multiply.c:82]	3 Assumed d...	2.023s	2.023s	Scalar	vector dependence ...					Float3
[loop in matvec at Multiply.c:69]	2 Potential ...	0.765s	0.765s	Vectorized (B...		SSE	~25%	0.99x	4	Unpacks
[loop in matvec at Multiply.c:60]	3 Ineffective ...	0.679s	0.679s	Vectorized Vers...		SSE	~99%	3.96x	4	Float3
[loop in matvec at Multiply.c:49]		3.919s	0.451s	Scalar	inner loop was alre...					Shuffles
f matvec		3.935s	0.016s	Function						Shuffles; Unpacks
f _schr_common_main_seh		3.949s	0.000s	Function						Float3
[loop in main at Driver.c:155]	1 Data type c...	3.935s	0.000s	Scalar	loop with function ...					Type Conversions
f main	1 Data type c...	3.949s	0.000s	Function						Shifts; Shuffles; Type ...
f printf		0.014s	0.000s	Function						Float3
f _stdio_common_vfprintf		0.014s	0.000s	Function						
f _crt_seh_guarded_call<int>::operator()<<lam		0.014s	0.000s	Function						
f <lambda_36eeb330e99a4f0bf8a7da86d0894cb>		0.014s	0.000s	Function						
f _crt_stdio_end_temporary_buffering_select		0.014s	0.000s	Function						

Create a Read-only snapshot

Click the  icon and save a **snapshot_noalias** result.

Generate Instructions for the Highest Instruction Set Architecture

Generating code for different instruction sets available on your compilation host processor may improve performance.

The `QxHost` (Windows OS) and `xHost` (Linux OS) options tell the compiler to generate instructions for the highest instruction set available on the compilation host processor.

To see if the `QxHost` and `xHost` options improve performance, do the following:

On Linux OS

From the same terminal window, build the application:

1. Navigate to the `vec_samples/` directory.
2. Rebuild the target application as follows:

```
make xhost
```

The command builds the application with the following compiler options: `-g -D NOALIAS -xHost`.

On Windows OS

From the same command prompt window:

1. Navigate to the `vec_samples/` directory.
2. Rebuild the target application as follows:

```
build.bat xhost
```

The script builds the application with the following compiler options: `/O2 /Qstd=c99 /fp:fast /Isrc /Zi /Qopenmp /DNOALIAS /QxHost`.

Re-run the Vectorization and Code Insights perspective from GUI or CLI.

Run Vectorization and Code Insights from GUI

1. Open the project in GUI:

```
advisor-gui .\vec_samples
```

2. In the **Analysis Workflow** pane for the Vectorization and Code Insights perspective, set data collection accuracy level to **Medium**.

At this accuracy level, Intel Advisor collects Survey and Characterization (Trip Counts) data.

3. Run the perspective.

Run Vectorization and Code Insights from CLI

On Linux OS

From the same terminal window:

1. Collect Survey data using the following command:

```
advisor --collect=survey --project-dir=./results -- ./vec_samples
```

2. Collect Trip Counts data using the following command:

```
advisor --collect=tripcounts --project-dir=./results -- ./vec_samples
```

When the analysis execution completes, the `vec_samples` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

On Windows OS

From the same command prompt window:

1. Collect Survey data using the following command:

```
advisor --collect=survey --project-dir=./results -- vec_samples.exe
```

2. Collect Trip Counts data using the following command:

```
advisor --collect=tripcounts --project-dir=./results -- vec_samples.exe
```

When the analysis execution completes, the `vec_samples` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

View the Results

If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

If you collect data using CLI, open the results in GUI using the following command:

```
advisor-gui ./results
```

If the result does not open automatically, click **Show Result**.

Check the changes in the **Summary** and open the **Survey Report** to assess the changes in application performance. In the report, notice the following:

- The **Elapsed time** probably improves.
- The values in the **Vector ISA** and **VL** columns in the top pane (probably) change.

Create a Read-only Snapshot

Click the  icon and save a **snapshot_xhost** result.

Next Steps

1. Pay attention to data dependencies assumed by the compiler and check, whether these dependencies are real and prevent your functions/loops from vectorizing. To do that, run the **Dependencies** analysis, mark the loops containing proven dependencies, and rebuild the application adding `/DREDUCTION` (Windows OS) and `-D REDUCTION` (Linux OS) compiler options.
2. Eliminate issues leading to significant vector code execution slowdown or block automatic vectorization by the compiler. To do that, run the **Memory Access Patterns** and modify memory access patterns in the problematic functions/loops.
3. Align data to assist automatic vectorization. For details, see [Data Alignment to Assist Vectorization](#).
4. Reorganize code to inline loops and enable the compiler to tell which variables you want to process and determine that vectorization is safe.

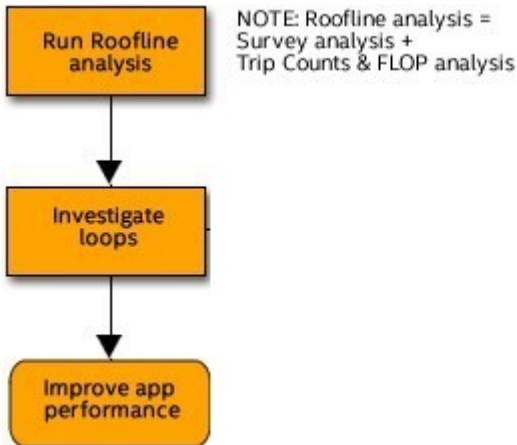
See Also

- [User Guide: Analyze Vectorization Perspective](#)
- [Vectorization Resources for Intel Advisor Users](#)

- [Cookbook: Analyze Vectorization and Memory Aspects of an MPI Application](#)

Identify Performance Bottlenecks Using CPU Roofline

CPU / Memory Roofline Insights perspective enables you to visualize actual performance against hardware-imposed performance ceilings, as well as determine the main limiting factor (memory bandwidth or compute capacity).



There are two ways to run the CPU / Memory Roofline Insights perspective: from the Intel® Advisor GUI and from CLI. Intel Advisor enables you to open results collected using both methods in the GUI.

Run CPU / Memory Roofline Insights Perspective from Intel® Advisor GUI

In the **Analysis Workflow** pane, the drop-down menu to select the CPU / Memory Roofline Insights

perspective, set data collection accuracy level to **Low**, and click the  button to run it. At this accuracy level, Intel Advisor:

- Measures the hardware limitations of your machine and collects loop/function timings using the **Survey analysis**.
- Collects floating-point and integer operations data, and memory data using the **Characterization analysis**.

For details about data collection accuracy presets, see [Intel Advisor User Guide: CPU Roofline Accuracy Presets](#). Upon completion, Intel Advisor displays a **Roofline** chart.

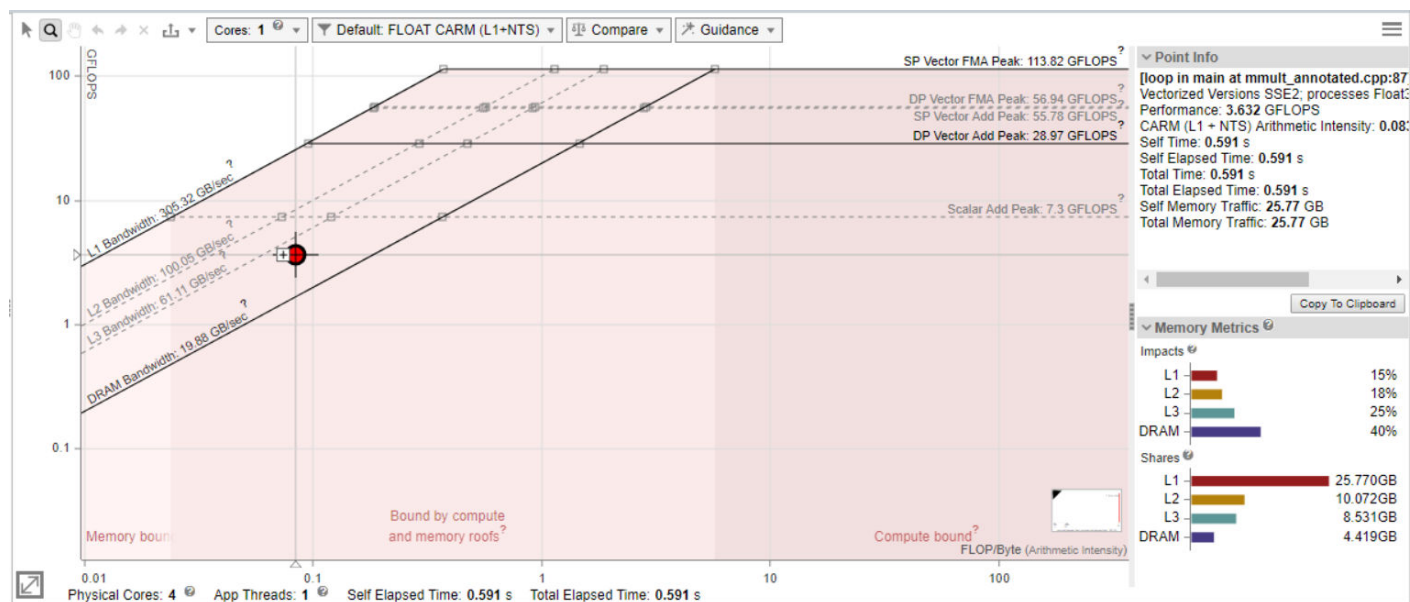
The **Roofline chart** plots an application's achieved performance and arithmetic intensity against the machine's **maximum achievable performance**:

- Arithmetic intensity (x axis) - measured in number of floating-point operations (FLOPs) and/or integer operations (INTOPs) per byte, based on the loop/function algorithm, transferred between CPU/VPU and memory.
- Performance (y axis) - measured in billions of floating-point operations per second (GFLOPS) and/or billions of integer operations per second (GINTOPS).

In general:

- **Dots** of different color and size represent functions/loops. The size and color of a dot represent execution time for this loop/function in relation to total execution time of the application. Large red dots are profitable to optimize as they take the longest execution time. Small green dots take less time and may be poor candidates for optimization.

- **Diagonal lines** indicate memory bandwidth limitations preventing loops/functions from achieving better performance without optimization. For example, the **L1 Bandwidth** roofline represents the maximum amount of work that can get done at a given arithmetic intensity if the loop always hits L1 cache. A loop does not benefit from L1 cache speed if a dataset causes it to miss L1 cache too often. In this case, it is subject to the limitations of the lower-speed L2 cache it is hitting. So, a dot representing a loop that misses L1 cache too often but hits L2 cache is positioned below the **L2 Bandwidth** roofline.
- **Horizontal lines** indicate compute capacity limitations preventing loops/functions from achieving better performance without optimization. For example, the **Scalar Add Peak** represents the peak number of add instructions that can be performed by a scalar loop under these circumstances. The **Vector Add Peak** represents the peak number of add instructions that can be performed under these circumstances by a vectorized loop with the highest instruction set available. So, a dot representing a loop that is not vectorized is positioned somewhere below the **Scalar Add Peak** roofline.
- A dot cannot exceed the topmost rooflines, as these represent the maximum capabilities of the machine; however, not all loops can utilize maximum machine capabilities.
- The greater the distance between a dot and the highest achievable roofline, the more room for optimization a function/loop has.



Run CPU / Memory Roofline Insights Perspective from Command Line Interface

To run CPU / Memory Roofline Insights perspective using `advisor` command line interface, use the following command:

```
advisor --collect=roofline --project-dir=./advi --search-dir src:p=./advi -- myApplication
```

This command is a batch mode that runs two analyses one by one:

1. Survey analysis that collects loops/functions execution time data.
2. Characterization analysis that collects floating-point and integer operations, memory traffic and mask utilization metrics for AVX-512 platforms to measure arithmetic intensity and performance of your application, and compute capacity of your hardware.

To view the achieved performance of your application against hardware-imposed performance ceilings on an interactive Roofline chart, open the collected results in the Intel Advisor GUI or use the following command to generate an interactive HTML Roofline report:

```
advisor --report=roofline --report-output=./advi/advisor-roofline.html --project-dir=./advi
```

Where `report-output` option specifies the directory and the HTML file into which Intel Advisor saves the generated report.

For details about generating CLI reports, see the respective section in the [Intel Advisor User Guide](#) or use the following command in your terminal:

```
advisor --help report
```

Intel Advisor enables you to create a read-only result snapshot using the following command:

```
advisor --snapshot --project-dir=./advi --pack --cache-sources --cache-binaries -- /tmp/  
my_proj_snapshot
```

What's Next

If one or more loops is not vectorizing properly and performance is unsatisfactory:

1. Consider working with the most time-consuming function/loop indicated on a Roofline chart.
 - Use the **Code Analytics** tab to examine the main information for the selected function/loop. Refer to the **Roofline** pane to identify whether the function/loop is compute or memory bound.
 - Use **Recommendations** tab to view hints on possible optimization steps for the selected function/loop in the **Roofline Guidance** section.
2. If your loop is compute bound:
 - Check the **Vectorized Loops/Efficiency** values in the **Survey Report**.
 - Consider running [Dependencies](#) analysis to discover why the compiler assumed a dependency and did not vectorize the selected function/loop.
 - Consider running [Memory Access Patterns \(MAP\)](#) analysis to identify expensive memory instructions.
3. If your loop is memory bound:
 - [Examine Relationships Between Memory Levels](#)

See Also

- Explore the common use cases described in Intel Advisor Cookbook:
 - [Identify Bottlenecks Iteratively: Cache-Aware Roofline](#)
 - [Optimize Memory Access Patterns using Loop Interchange and Cache Blocking Techniques](#)
- Explore useful [Roofline Resources for Intel Advisor Users](#).

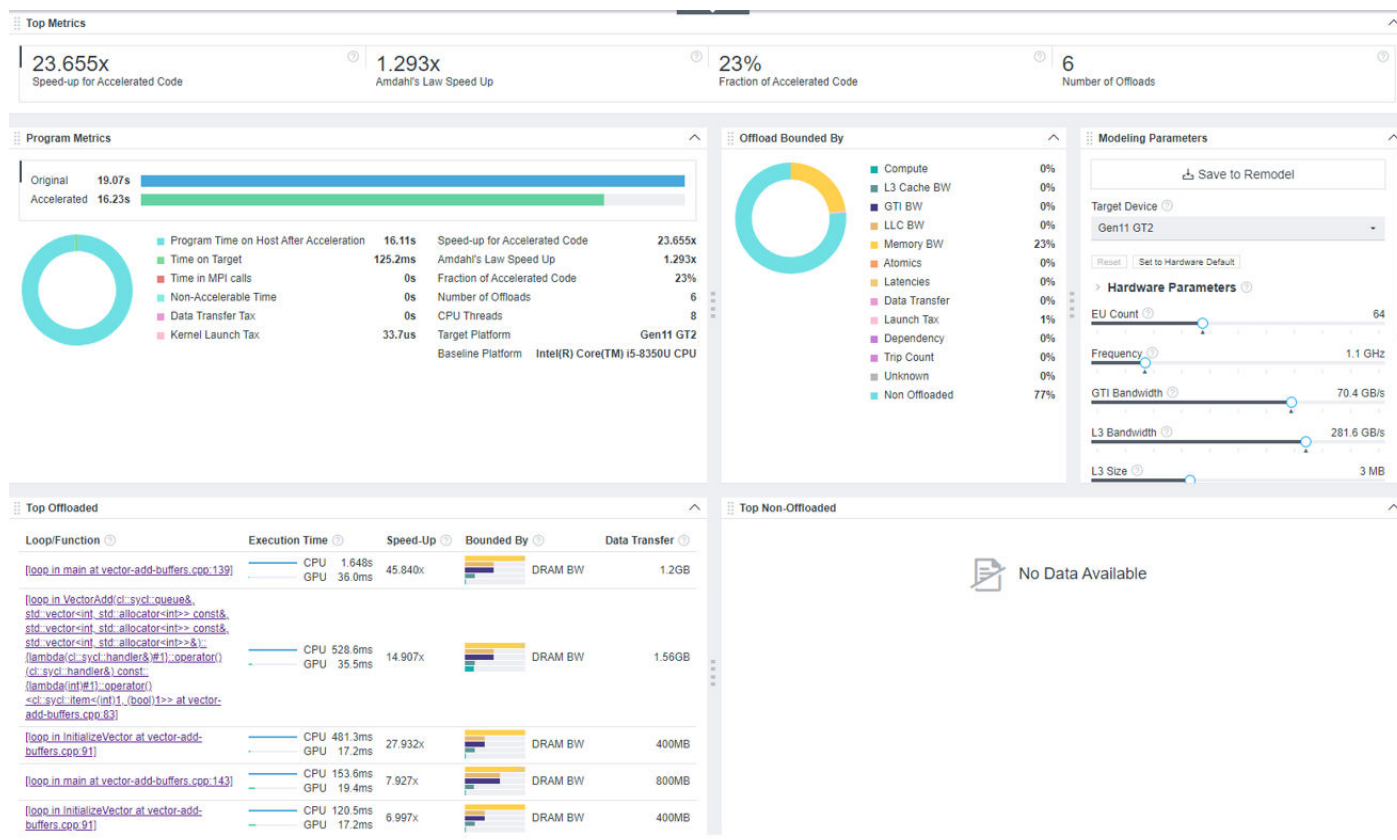
Identify High-impact Opportunities to Offload to GPU

With the Offload Modeling perspective of the Intel® Advisor, you can model performance of your application on a target graphics processing unit (GPU) device and identify code regions that are the most profitable to run on the target GPU.

Offload Modeling perspective has two workflows:

- With CPU-to-GPU modeling, you can profile an application running on a CPU and model its performance on a target GPU device to determine if you should offload parts of your application to the GPU.
- With GPU-to-GPU modeling, you can profile an application running on a GPU and model its performance on a different GPU device to estimate a potential speedup from running your application on the different target.

This page explains how to profile a `vector-add` sample with CPU-to-GPU modeling and estimate its speedup on a target GPU.



Follow the steps:

1. Build your application.
2. Model application performance running on a GPU.
3. Examine application performance estimated on the GPU.

Prerequisites

1. Download the `vector-add` code sample from oneAPI samples GitHub* repository.
You can also use your own application to follow the instructions below.
2. Install the Intel Advisor as a [standalone](#) or as [part of Intel® oneAPI Base Toolkit](#). For installation instructions, see [Install Intel Advisor](#) in the user guide.
3. Install the Intel® oneAPI DPC++/C++ Compiler as a [standalone](#) or as [part of Intel® oneAPI Base Toolkit](#). For installation instructions, see [Intel® oneAPI Toolkits Installation Guide](#).
4. Set up environment variables for the Intel Advisor and Intel oneAPI DPC++/C++ Compiler. For example, run the `setvars` script in the installation directory.

This document assumes you installed the tools to a [default location](#). If you installed the tools to a different location, make sure to replace the default path in the commands below.

Important Do not close the terminal or command prompt after setting the environment variables. Otherwise, the environment resets.

Build Your Application

On Linux* OS

From the terminal where you set the environment variables:

1. Navigate to the `vector-add` sample directory.
2. Compile the application using the following command:

```
icpx -g -fiopenmp -fopenmp-targets=spir64 -fsycl src/vector-add-buffers
```

3. Run the application with a vector size 100000000 to verify the build as follows:

```
./vector-add-buffers 100000000
```

If the application is built successfully, you should see the output similar to the following:

```
Vector size: 100000000
[0]: 0 + 0 = 0
[1]: 1 + 1 = 2
[2]: 2 + 2 = 4
...
[99999999]: 99999999 + 99999999 = 199999998
Vector add successfully completed on device.
```

The `vector-add-buffers` application uses SYCL and runs on a GPU by default. For the workflow in the topic, you should temporarily offload it to a CPU to analyze as described in the section below.

On Windows* OS

From the command prompt where you set the environment variables:

1. Navigate to the `vector-add` sample directory.
2. Compile the application using the following command:

```
icx-cl -fsycl /O2 /EHsc /Zi -o vector-add-buffers.exe src/vector-add-buffers.cpp
```

3. Run the application with a vector size 100000000 to verify the build as follows:

```
vector-add-buffers.exe 100000000
```

If the application is built successfully, you should see the output similar to the following:

```
Vector size: 100000000
[0]: 0 + 0 = 0
[1]: 1 + 1 = 2
[2]: 2 + 2 = 4
...
[99999999]: 99999999 + 99999999 = 199999998
Vector add successfully completed on device.
```

The `vector-add-buffers` application uses SYCL and runs on a GPU by default. For the workflow in the topic, you should temporarily offload it to a CPU to analyze as described in the section below.

Model Application Performance Running on a GPU

Run Offload Modeling from Graphical User Interface (GUI)

1. From the terminal or command prompt where you set the environment variables, launch the Intel Advisor GUI:

```
advisor-gui
```

2. Create a `vector-add` project for the `vector-add-buffers` application. Follow the instructions in the [Before You Begin](#) page. When you have the **Project Properties** dialog box open:
 - a. Go to the **Analysis Target** tab > **Survey Analysis Types** > **Survey Hotspots Analysis**.
 - b. Click **Browse...** near the **Application** field and navigate to the `vector-add-buffers` application. Click **Open**.
 - c. To set the vector size, in the **Application parameters** field, type 100000000.

- d. To offload the application to a CPU temporarily, click **Modify...** near the **User-defined environment variables** field. The **User-defined Environment Variables** dialog box opens.
 - e. Click the empty line in the **Variable** column and enter variable name `SYCL_DEVICE_FILTER`.
 - f. Click the empty line in the **Value** column and enter variable value `opencl:cpu`.
 - g. Click **OK** to save the changes.
 - h. Go to the **Analysis Target** tab > **Survey Analysis Types** > **Trip Counts and FLOP Analysis** and make sure the **Inherit Settings from Survey Hotspots Analysis Type** checkbox is selected.
 - i. Click **OK** to create the project.
3. In the **Perspective Selector** window, choose the Offload Modeling perspective.
 4. In the **Analysis Workflow** pane, select the following parameters:
 - a. Make sure the baseline device is set to **CPU**.
 - b. Set accuracy to **Medium**.
 - c. Select the **Gen11 GT2** target device.

5.



Click button to run the perspective.

During the perspective execution with medium accuracy, Intel Advisor:

- Analyzes your application total execution time and execution time of its loops/functions using the Survey analysis
- Counts how many iterations each cycle performs during application runtime using Characterization analysis
- Estimates execution time for code regions that can be offloaded to a target GPU and total time to transfer data from a CPU to a target GPU using the Performance Modeling analysis

When the execution completes, the Offload Modeling result is opened automatically.

Run Offload Modeling from Command Line Interface (CLI)

On Linux OS

From the command prompt where you set the environment variables:

1. Navigate to the `vector-add` sample directory.
2. Temporarily offload the application to a CPU with the `SYCL_DEVICE_FILTER` environment variable as follows:

```
export SYCL_DEVICE_FILTER=opencl:cpu
```

3. Run the Offload Modeling perspective with the medium accuracy level using the command-line collection preset:

```
advisor --collect=offload --config=gen11_icl --project-dir=./vector-add -- vector-add-buffers 100000000
```

This command runs the Offload Modeling perspective analyses for the default medium accuracy level one by one. During the perspective execution, Intel Advisor:

- Analyzes your application total execution time and execution time of its loops/functions using the Survey analysis
- Counts how many iterations each cycle performs during application runtime using Characterization analysis
- Estimates execution time for code regions that can be offloaded to a target GPU and total time to transfer data from a CPU to a target GPU using the Performance Modeling analysis

When the execution completes, the `vector-add` project is created automatically, which includes the Offload Modeling results. You can view them with a preferred method.

On Windows OS

From the command prompt where you set the environment variables:

1. Navigate to the `vector-add` sample directory.
2. Temporarily offload the application to a CPU with the `SYCL_DEVICE_FILTER` environment variable as follows:

```
set SYCL_DEVICE_FILTER=opencl:cpu
```

3. Run the Offload Modeling perspective with the medium accuracy level using the command-line collection preset:

```
advisor --collect=offload --config=gen11_icl --project-dir=.\vector-add -- vector-add-  
buffers.exe 100000000
```

This command runs the Offload Modeling perspective analyses for the default medium accuracy level one by one. During the perspective execution, Intel Advisor:

- Analyzes your application total execution time and execution time of its loops/functions using the Survey analysis
- Counts how many iterations each cycle performs during application runtime using Characterization analysis
- Estimates execution time for code regions that can be offloaded to a target GPU and total time to transfer data from a CPU to a target GPU using the Performance Modeling analysis

When the execution completes, the `vector-add` project is created automatically, which includes the Offload Modeling results. You can view them with a preferred method.

Examine Application Performance Estimated on the GPU

If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

If you collect data using CLI, open the results in GUI using the following command:

```
advisor-gui ./vector-add
```

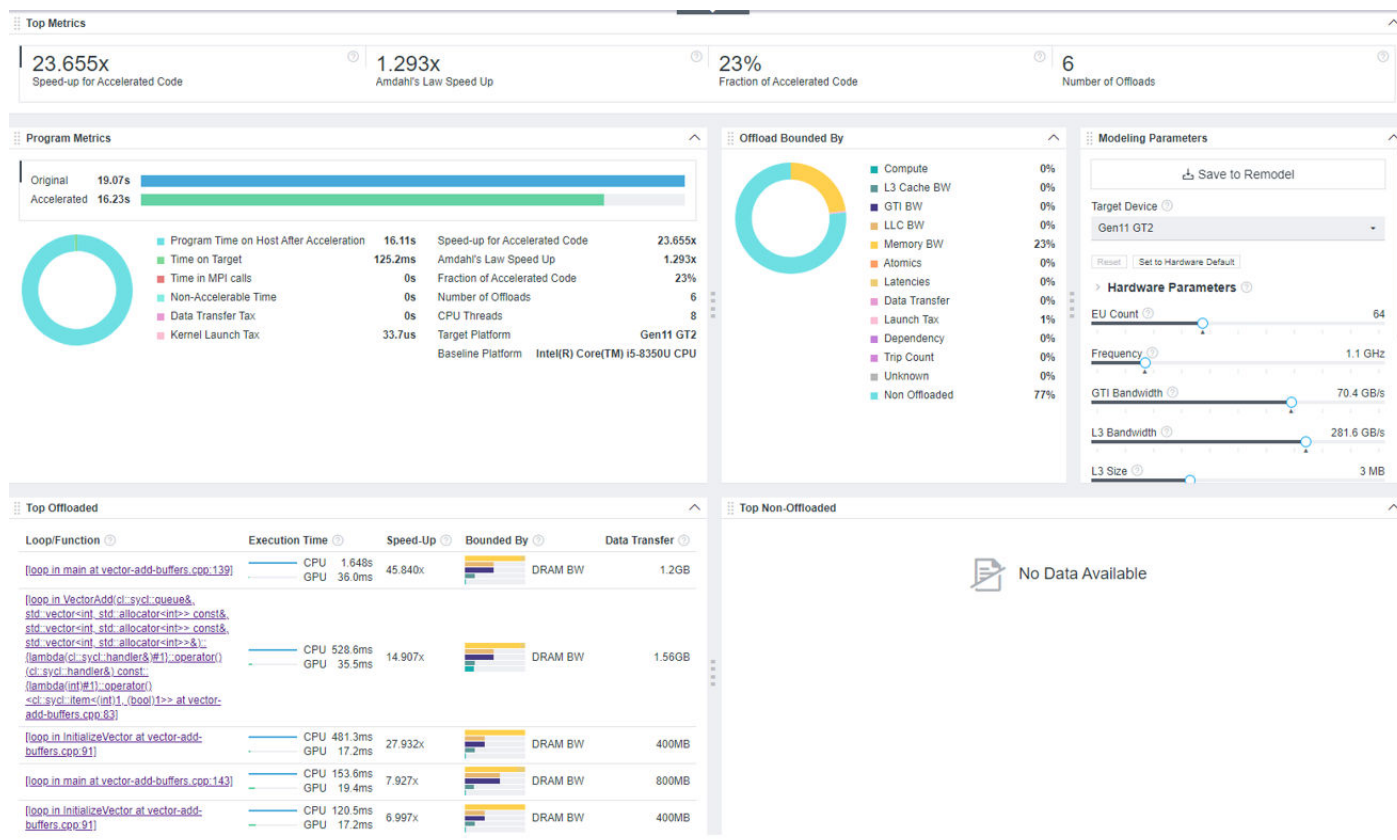
If the result does not open automatically, click **Show Result**.

You can also view the results in an interactive HTML report. The report data and structure are similar to the GUI results. You can share this report or open it on a remote system even if it does not have Intel Advisor installed. See [Work with Standalone HTML Reports](#) in the Intel Advisor User Guide for details.

NOTE The results that you see when you open the report might be different from what is shown in the following sections due to a different baseline device or system characteristics. You can still review the sections to understand the result analysis workflow in general.

Explore Performance Estimations for the Whole Application

When you open the Offload Modeling result in GUI, Intel Advisor shows the **Summary** tab first. This window is a dashboard containing the main information about application execution measured on a baseline device and estimated on a target GPU, estimated speedup, and more.



In the **Summary** window, notice the following:

1. As the **Top Metrics** pane shows, speedup estimated for offloaded parts of the `vector-add-buffers` code *only* is 23.655x, but the speedup calculated by the Amdahl's law, which is the speedup for the whole application, is only 1.293x. This means that although the offloaded code regions can run faster on the GPU, they do not have a big impact on the overall application speedup. It might be because only 23% of the code is recommended for offloading.
2. Explore per-application metrics in the **Program Metrics** pane. The estimated time on the target GPU (accelerated time) is 16.23 seconds, which includes 16.11 seconds on a host device and 125.2 milliseconds on the target device.
3. Investigate the factors that limit your offloaded code regions in the **Offload Bounded By** pane. This pane characterizes all potential bottlenecks for the whole application, such as compute, data transfer, and latencies, with percentage of code impacted.

The `vector-add-buffers` application has 77% of non-offloaded code. The offloaded code is mostly bounded by host memory bandwidth (23%), which means that the application might be memory-bound after offloading as it underutilizes memory resources on the host device. If you optimize how the application uses memory after offloading or increase memory bandwidth, you might improve application performance.

You can see the bottlenecks per code region in the **Top Offloaded** pane by hovering over a histogram in the **Bounded By** column.

4. In the **Hardware Parameters** pane, notice the host memory bandwidth for the current modeled Gen11 GT2 device is 48 GB/s.

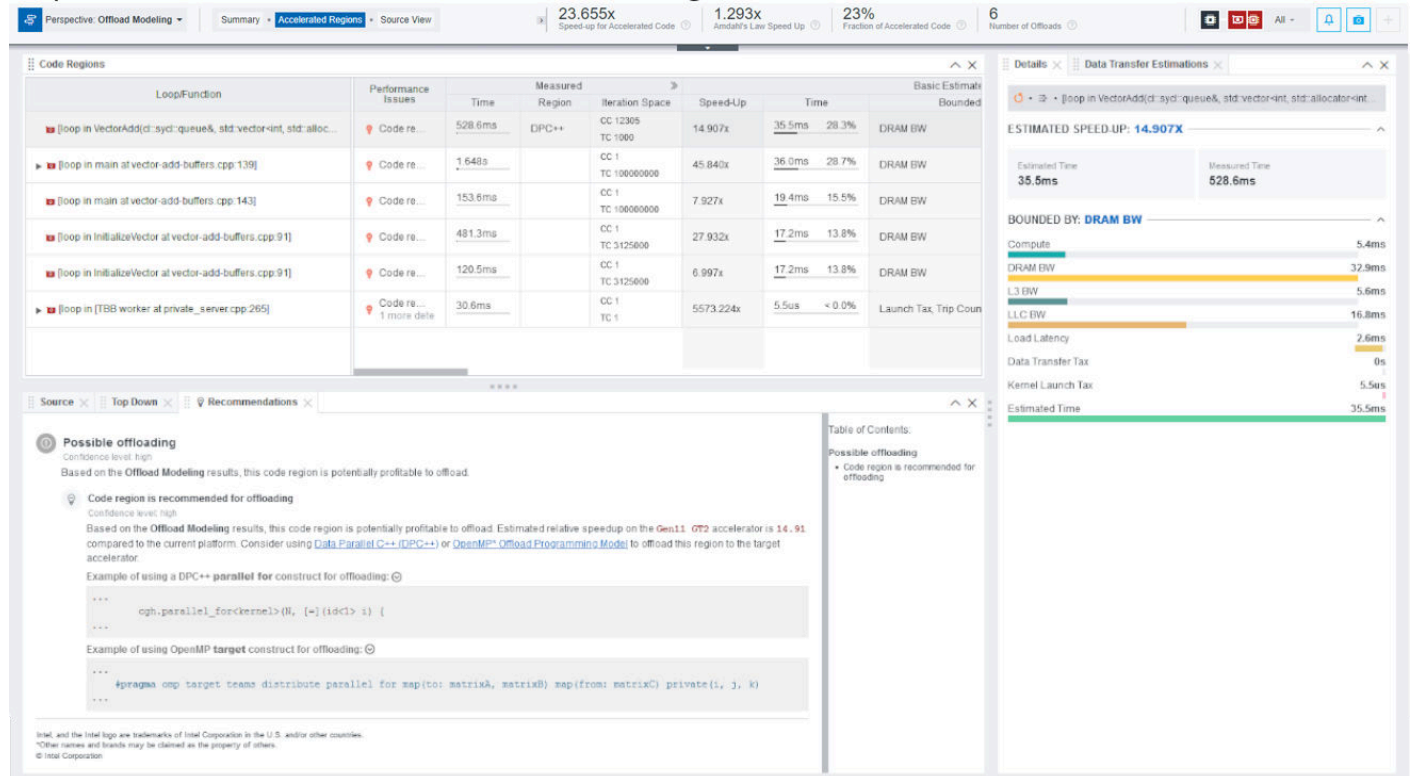
You can move the **Memory BW** slider in the pane to increase the value and remodel application performance for a custom device with higher memory bandwidth from command line. You may get a better estimated performance.

You can use this pane to adjust GPU parameters to model performance on a different device with custom parameters.

- Examine the loops/functions with the highest speedup achieved by offloading to a GPU in the **Top Offloaded** pane. The topmost loop *[loop in main at vector-add-buffers.cpp:139]* has the highest estimated speedup on the target. You can click a loop to switch to the **Accelerated Regions** tab and explore the loop performance on the target GPU in more detail.

Analyze Estimated Performance for Specific Loops

In the **Accelerated Regions** tab, you can analyze estimated performance for each loop and check what loops are recommended to be offloaded to the target GPU.



Intel Advisor reports 5 loops/functions as potentially profitable to be offloaded to the target GPU.

- In the bottom pane, switch to the **Source** tab to examine code region sources.
- Click each loop in the **Code Regions** pane and examine its source in the **Source** tab to understand code region purpose.
 - [loop in main at vector-add-buffers.cpp:139]* is a loop that is supposed to run on a host device to be compared with the execution result of the main loop.
 - [loop in VectorAdd(...)<...> at vector-add-buffers.cpp:83]* is the main loop that adds vectors. It is a SYCL kernel.
 - [loop in main at vector-add-buffers.cpp:143]* is a loop that is supposed to run on a host device to compare the results of the *[loop in VectorAdd(...)<...> at vector-add-buffers.cpp:83]* and *[loop in main at vector-add-buffers.cpp:139]*.
 - [loop in InitializeVector at vector-add-buffers.cpp:91]* is a loop that initializes data.
 - [loop in [TBB worker at private_server.cpp:265]* is a subordinate oneAPI Threading Building Blocks loop.

Based on application source analysis, *[loop in VectorAdd(...)<...> at vector-add-buffers.cpp:83]* is the only offload candidate.

3. Click the `[loop in VectorAdd(...)<...> at vector-add-buffers.cpp:83]` loop in the **Code Regions** pane to analyze its performance in more details and examine the following metrics:
 - a. The original time *measured* on a baseline CPU device is 528.6 milliseconds (**Measured** column group), time *estimated* on the target GPU after offload is 36 milliseconds (**Basic Estimates Metrics** column group), which is 14.907% of the total estimated time on the target GPU. This means that the code region can run 45.840x times faster on the target as reported in the **Speed-Up** column in **Basic Estimates Metrics**.
 - b. In the **Summary** window, you saw that the application is mostly memory-bound on the target GPU. As indicated in the **Bounded By** column (**Basic Estimated Metrics**), the selected code region is bounded by *DRAM bandwidth*.
 - c. Scroll to the right to the **Estimated Bounded By** column group. As indicated in the **Throughout** column, the code region spends 32.9 milliseconds to read from and write to the DRAM memory.
 - d. Scroll to the **Estimated Data Transfer with Reuse** to see the estimated data traffic modeled between host and target devices.
 - e. Scroll to the right to the **Memory Estimations** column group to see the traffic between target device memory levels modeled with cache simulation analysis. The column group shows traffic for each memory level on the target, such as L3 cache, last-level cache (LLC), DRAM memory, GTI memory.
4. Examine performance summary for the selected code region in the **Details** pane on the right. As the pane reports, total estimated time for the code region is *36 milliseconds*, which includes:
 - 5.4 milliseconds for computing vector addition
 - 32.9 milliseconds for reading from and writing to DRAM
 - 5.6 milliseconds for reading from and writing to L3 cache
 - 16.8 milliseconds for writing to last level cache (LLC)

This means the code region mostly spends time for working with memory.
5. View actionable recommendations for the selected loop in the **Recommendations** tab. Intel Advisor recommends you offload the selected loop to the target as it has high estimated speedup. The recommendation includes example code snippets for offloading the loop using SYCL or OpenMP*, which you can expand to see more syntax.

You can also view the selected loop in using the **Top-Down** tab to locate it in the application call tree and see how it is related to other loops and examine its source code using the **Source** tab

Continue to examine the performance of other code regions in the application to get a better understanding on its performance estimated on the target.

Next Steps

1. In the **Code Regions** pane, notice that `[loop in VectorAdd(...)<...> at vector-add-buffers.cpp:83]` has *Parallel: Explicit* dependency type, but there are three other loops/functions that have *Parallel: Assumed* dependency type reported (**Measured** column group). This means Intel Advisor does not have information about dependencies in these code regions, but it assumes there are no dependencies and the loops/functions can be parallelized.

In most cases, if a loop has dependencies, it is not recommended to offload it to the target GPU. You can analyze the loop-carried dependencies with the Dependencies analysis of the Intel Advisor. See [Check How Assumed Dependencies Affect Modeling](#) in the user guide for a recommended workflow.
2. If Intel Advisor detects real loop-carried dependencies, it models them as sequential kernels. Usually, such kernels are not profitable to run on a GPU. You need to resolve the dependencies before you offload your code to the target.
3. Get more detailed report about data transfers in the application by running the Characterization analysis with a different data transfer simulation mode: **medium** to understand how memory objects are transferred or **full** to check if data reuse can improve the application performance.

Based on the data reported about the `vector-add-buffers` sample application, you can run it on the target GPU and optimize its performance or you can run other analyses to learn more about application behavior on the target GPU. After this, you can use [GPU Roofline Insights perspective](#) to profile application actual performance on the target GPU and see how it uses hardware resources.

See Also

- [User Guide: Offload Modeling](#)
- [Cookbook: Model GPU Application Performance for a Different GPU Device](#)
- [Cookbook: Identify Code Regions to Offload to GPU and Visualize GPU Usage](#)
- [Offload Modeling Resources for Intel Advisor Users](#)

Measure GPU Performance Using GPU Roofline

GPU Roofline Insights perspective enables you to estimate and visualize actual performance of GPU kernels using benchmarks and hardware metric profiling against hardware-imposed performance ceilings, as well as determine the main limiting factor.

There are two ways to run GPU Roofline Insights perspective: from the Intel® Advisor GUI and from CLI. Intel Advisor enables you to open results collected using both methods in the GUI.

Run GPU Roofline Insights Perspective from Intel® Advisor GUI

In the **Analysis Workflow** pane, use a drop-down menu to select the GPU Roofline Insights perspective, set

data collection accuracy level to **Low**, and click the  button to run it. At this accuracy level, Intel Advisor:

- Measures the hardware limitations and collects OpenCL™, OpenMP*, oneAPI Level Zero (Level Zero) and SYCL kernels timings and memory data using the **Survey analysis** with GPU profiling.
- Collects floating-point and integer operations data using the **Trip Counts and FLOP analysis** with GPU profiling.

For details about data collection accuracy presets, see [Intel Advisor User Guide: GPU Roofline Accuracy Presets](#). Upon completion, Intel Advisor displays a **GPU Roofline Summary**. Switch to the **GPU Roofline Regions** tab to view the **Roofline Chart** and identify the main factors limiting the performance of your application.

Important

GPU profiling is applicable only to Intel® Processor Graphics.

A **Roofline** chart plots an application's *achieved performance* and *arithmetic intensity* against the machine's *maximum achievable performance*:

- Arithmetic intensity (x axis) - measured in number of floating-point operations (FLOPS) per byte for FLOAT Roofline chart and in number of integer operations (INTOPS) per byte for INT Roofline chart based on the kernel algorithm, transferred between GPU and memory
- Performance (y axis) - measured in billions of floating-point operations (GFLOPS) per second for FLOAT Roofline chart and in billions of integer operations (GINTOPS) per second for INT Roofline chart

In general:

- **Dots** represent kernels. The size and color of each dot represent relative execution time for each kernel. Large red dots take the most time, so are the best candidates for optimization. Small green dots take less time, so may not be worth optimizing.

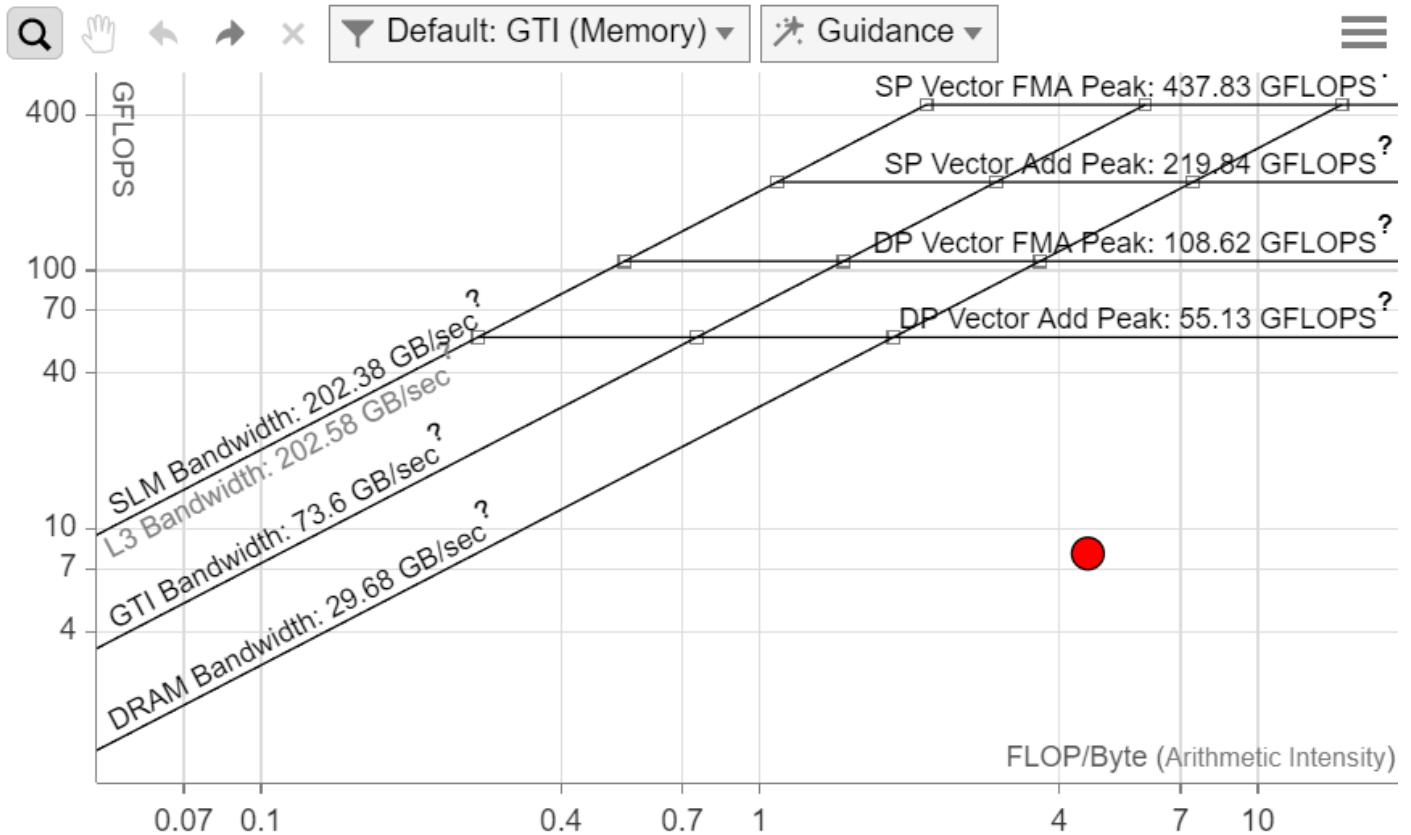
- **Diagonal lines** indicate *memory bandwidth limitations* preventing kernels from achieving better performance without some form of optimization.

Depending on your system configuration the following rooflines might be available on the Roofline chart:

- **L3 cache roof:** Represents the maximal bandwidth of the L3 cache for your current graphics hardware. Measured using an optimized sequence of load operations, iterating over an array that fits entirely into L3 cache.
- **SLM cache roof:** Represents the maximal bandwidth of the Shared Local Memory for your current graphics hardware. Measured using an optimized sequence of load and store operations that work only with SLM.
- **GTI roof:** Represents the maximum bandwidth between the GPU and the rest of the SoC. This estimate is calculated via analytical formula based on the maximum frequency of your current graphics hardware.
- **DRAM roof:** Represents the maximal bandwidth of the DRAM memory available to your current graphics hardware. Measured using an optimized sequence of load operations, iterating over an array that does not fit in GPU caches.
- **Horizontal lines** indicate *compute capacity limitations* preventing kernels from achieving better performance without some form of optimization.
- A dot cannot exceed the topmost rooflines, as these represent the maximum capabilities of the machine. However, not all kernels can utilize maximum machine capabilities.
- The greater the distance between a dot and the highest achievable roofline, the more opportunity exists for performance improvement.

The GPU Roofline chart is based on a CPU Roofline chart layout, but there are some differences:

- The dots on the chart correspond to *OpenCL, OpenMP, Level Zero and SYCL kernels*, while in the CPU version, they correspond to individual loops.
- Some displayed information and controls (for example, thread/core count) are not relevant to GPU Roofline. For more information, see the table below.
- The GPU Roofline chart enables you to view arithmetic intensity of one kernel at multiple memory levels. To do so, double-click a dot representing this kernel or select it and press ENTER. The dots that appear on the Roofline chart correspond to different memory levels used to calculate arithmetic intensity. Hover over a dot to identify its arithmetic intensity. To show or hide certain dots from a chart, use the **Memory Level** drop-down filter.



Run GPU Roofline Insights Perspective from Command Line Interface

To run GPU Roofline Insights perspective using `advisor` command line interface, use the following command:

```
advisor --collect=roofline --profile-gpu --project-dir=./advi --search-dir src:p=./advi --myApplication
```

1. Collect performance metrics for loops/functions of your application using Survey analysis:

```
advisor --collect=survey --profile-gpu --project-dir=./advi --search-dir src:p=./advi --myApplication
```

2. Collect floating-point operations data using Characterization analysis:

```
advisor --collect=tripcounts --no-trip-counts --flop --profile-gpu --project-dir=./advi --search-dir src:p=./advi -- myApplication
```

Where:

- `no-trip-counts` disables collection of trip counts during Characterization analysis.
- `flop` enables collection of data about floating-point and integer operations, memory traffic, and mask utilization metrics for AVX-512 platforms during Characterization analysis.

This command is a batch mode that runs two analyses one by one:

1. Survey analysis that collects loops/functions execution time data and measure L3, SLM, and GTI traffic.
2. Characterization analysis that collects floating-point and integer operations considering mask utilization, and CARM memory traffic to measure arithmetic intensity and performance of your application.

To view the achieved performance of your application against hardware-imposed performance ceilings on an interactive Roofline chart, open the collected results in the Intel Advisor GUI or use the following command to generate an interactive HTML Roofline report:

```
advisor --report=roofline --profile-gpu --report-output=./advi/advisor-roofline.html --project-dir=./advi
```

Where `report-output` option specifies the directory and the HTML file into which Intel Advisor saves the generated report.

By default, Intel Advisor generates a FLOAT Roofline chart. To switch to INT Roofline chart, add a `--data-type=int` option to your command.

For details about generating CLI reports, see the respective section in the [Intel Advisor User Guide](#) or use the following command in your terminal:

```
advisor --help report
```

Intel Advisor enables you to create a read-only result snapshot using the following command:

```
advisor --snapshot --project-dir=./advi --pack --cache-sources --cache-binaries -- /tmp/  
my_proj_snapshot
```

What's Next

Use the **GPU Roofline Summary** (available in GUI only) to compare performance of your application on a CPU and on a GPU device.

Investigate performance metrics for your kernels and recommendations with possible optimization steps in the **GPU Code Analytics** pane.

See Also

Explore a use case for optimizing GPU usage described in [Intel Advisor Cookbook: Identify Code Regions to Offload to GPU and Visualize GPU Usage](#).

Prototype Threading Designs

With the Threading perspective, you can identify the best candidates for parallelizing, prototype threading and check, if there are data dependencies preventing parallelizing of certain functions/loops.

This page explains how to profile `nqueens` application and choose the best candidates for parallelization with threads. You can also use your own application to follow the instructions below.

Summary Survey & Roofline Refinement Reports Annotation Report Suitability Report

Threading Perspective

Threading Perspective lets you analyze, design, tune, and check threading options without disrupting your development.

Program Metrics

Elapsed Time 5.39s Number of CPU Threads 1
Vector Instruction Set **None**

Performance Characteristics

Metrics	Total	
Total CPU time	5.00s	100%
Time in scalar code	5.00s	100%

> Vectorization Gain/Efficiency (Not Available)

Per Program Recommendations

Higher instruction set architecture (ISA) available
Consider recompiling your application using a higher ISA. [Show more](#)

Top Time-Consuming Loops

Consider adding parallel site and task annotations around these time-consuming loops found during Survey analysis.

Loop	Self Time	Total Time	Trip Counts
loop in setQueen at nqueens_serial.cpp:132	0.449s	4.996s	14
loop in solve at nqueens_serial.cpp:156	<0.001s	4.996s	14
loop in setQueen at nqueens_serial.cpp:103	1.934s	1.934s	4

Suitability And Dependencies Analysis Data

These annotated parallel sites were detected:

Site Location	Maximum Site Gain	Dependencies
loop in solve at nqueens_serial.cpp:154	6.4685634666754614	WAR:1 WAW:1

Recommendations

- align_loop_title loop in [setQueen](#) at [nqueens_serial.cpp:103](#)
- align_loop_title loop in [setQueen](#) at [nqueens_serial.cpp:132](#)

Follow the steps:

1. Unpack and build your application.
2. Collect baseline performance data.
3. Examine results to find opportunities for parallelization.
4. Mark best parallel opportunities with annotations.
5. Model threading parallelism.

Prerequisites

1. Install the Intel Advisor as a [standalone](#) or as [part of Intel® oneAPI Base Toolkit](#). For installation instructions, see [Install Intel Advisor](#) in the user guide.
2. Install the Intel® C++ Compiler Classic as a [standalone](#) or as [part of Intel® oneAPI HPC Toolkit](#). For installation instructions, see [Intel® oneAPI Toolkits Installation Guide](#).
3. Set up environment variables for the Intel Advisor and Intel® C++ Compiler Classic. For example, run the `setvars` script in the installation directory.

This document assumes you installed the tools to a [default location](#). If you installed the tools to a different location, make sure to replace the default path in the commands below.

Important Do not close the terminal or command prompt after setting the environment variables. Otherwise, the environment resets.

Unpack and Build Your Application

On Linux* OS

From the terminal where you set the environment variables:

1. Go to the `/opt/intel/oneapi/advisor/latest/samples/en/C++` directory.
2. Copy the `nqueens_Advisor.tgz` file to a writable directory or share on your system.
3. Extract the sample from the `.tgz` file.
4. Change directory to the `nqueens_Advisor/` directory in its unzipped location.
5. Build the sample application:

```
make 1_nqueens_serial
```

6. Run the application to verify the build:

```
./1_nqueens_serial
```

The application output window displays a board size of 14 and the total time it took to run the target.

On Windows* OS (From Command Line)

1. Find **Visual Studio Tools** for your Microsoft Visual Studio* and OS version, and select one of the command prompt shortcuts. For example, from the Microsoft Windows* 10 **Start** pane, select **Visual Studio 2019 > x64 Native Tools Command Prompt for VS2019**.
2. Go to the `C:\Program Files (x86)\Intel\oneAPI\advisor\latest\samples\en\C++` directory.
3. Copy the `nqueens_Advisor.zip` file to a writable directory or share on your system.
4. Extract the sample from the `.zip` file.
5. Change directory to the `nqueens_Advisor/` directory in its unzipped location.
6. Build the target in release mode:

```
devenv nqueens_Advisor.sln /build release /project 1_nqueens_serial
```

7. Change directory to the `Release` directory.
8. Run the application to verify the build:

```
1_nqueens_serial.exe
```

The application output window displays a board size of 14 and the total time it took to run the target.

On Windows* OS (From Microsoft Visual Studio)

1. Go to the `C:\Program Files (x86)\Intel\oneAPI\advisor\latest\samples\en\C++` directory.
2. Copy the `nqueens_Advisor.zip` file to a writable directory or share on your system.
3. Extract the sample from the `.zip` file.
4. Launch the Microsoft Visual Studio IDE.
5. Choose **File > Open > Project/Solution...**
6. In the **Open Project** dialog box, navigate to the `nqueens_Advisor/` directory in its unzipped location and open the `nqueens_Advisor.sln` file.

NOTE If you get a dialog window suggesting you to retarget the application, click **OK**.

7. If the **Solutions Configuration** drop-down is set to **Debug**, change it to **Release**.
8. Right-click the `1_nqueens_serial` project in the **Solution Explorer** and **Choose Set as Start Up Project**.
9. If you want to use the Intel® C++ Compiler Classic, right-click the `1_nqueens_serial` project and click **Intel Compiler > Use Intel C++ Compiler Classic**.
10. Right-click the `1_nqueens_serial` project, then choose **Properties** to verify the sample code uses the optimal release build settings.
For details about recommended build setting, see [Build Target Application](#).
11. Click the **OK** button to close the **Properties** dialog box.

12. Choose **Build > Clean Solution**.

13. Choose **Build > Build 1_nqueens_serial** to build the target.

The application output window displays a board size of 14 and the total time it took to run the target.

14. If the Visual Studio* IDE responds that any projects are out of date, click **No** to not build them.

Collect Baseline Performance Data

Run Threading Perspective from Graphical User Interface (GUI)

1. From the terminal or command prompt where you set the environment variables, launch the Intel Advisor GUI:

```
advisor-gui
```

2. Create a project for the just-built `vec_samples` application. For details, see [Before You Begin](#).

When in the **Project Properties** dialog box, make sure the **Inherit settings from Survey Hotspots Analysis Type** checkbox is selected in the **Trip Counts and FLOP Analysis, Dependencies Analysis**, and **Memory Access Patterns Analysis** types.

NOTE If you work in the Microsoft Visual Studio IDE, you do not need to create a project as the Intel Advisor creates it automatically when you first open the Intel Advisor GUI.

3. From the **Perspective Selector** pane, choose the Threading perspective.

4. In the **Analysis Workflow** pane, set data collection accuracy level to **Low**, and click the



button to run the perspective.

At this accuracy level, Intel Advisor runs Survey analysis to profile the application.

Run Threading from Command Line Interface (CLI) on Linux OS

Run Survey analysis to collect performance metrics and identify loops/functions with the longest total time:

```
advisor --collect=survey --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial
-- 1_nqueens_serial
```

NOTE In the Threading perspective, you should specify the source search directory using the `--search-dir` option.

When the analysis execution completes, the `1_nqueens_serial` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

Run Threading from CLI on Windows OS

Run Survey analysis to collect performance metrics and identify loops/functions with the longest total time:

```
advisor --collect=survey --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial
-- 1_nqueens_serial.exe
```

NOTE In the Threading perspective, you should specify the source search directory using the `--search-dir` option.

When the analysis execution completes, the `1_nqueens_serial` project is created automatically, which includes the Vectorization and Code Insights results. You can view them from Intel Advisor GUI.

Examine Results to Find Opportunities for Parallelization

If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

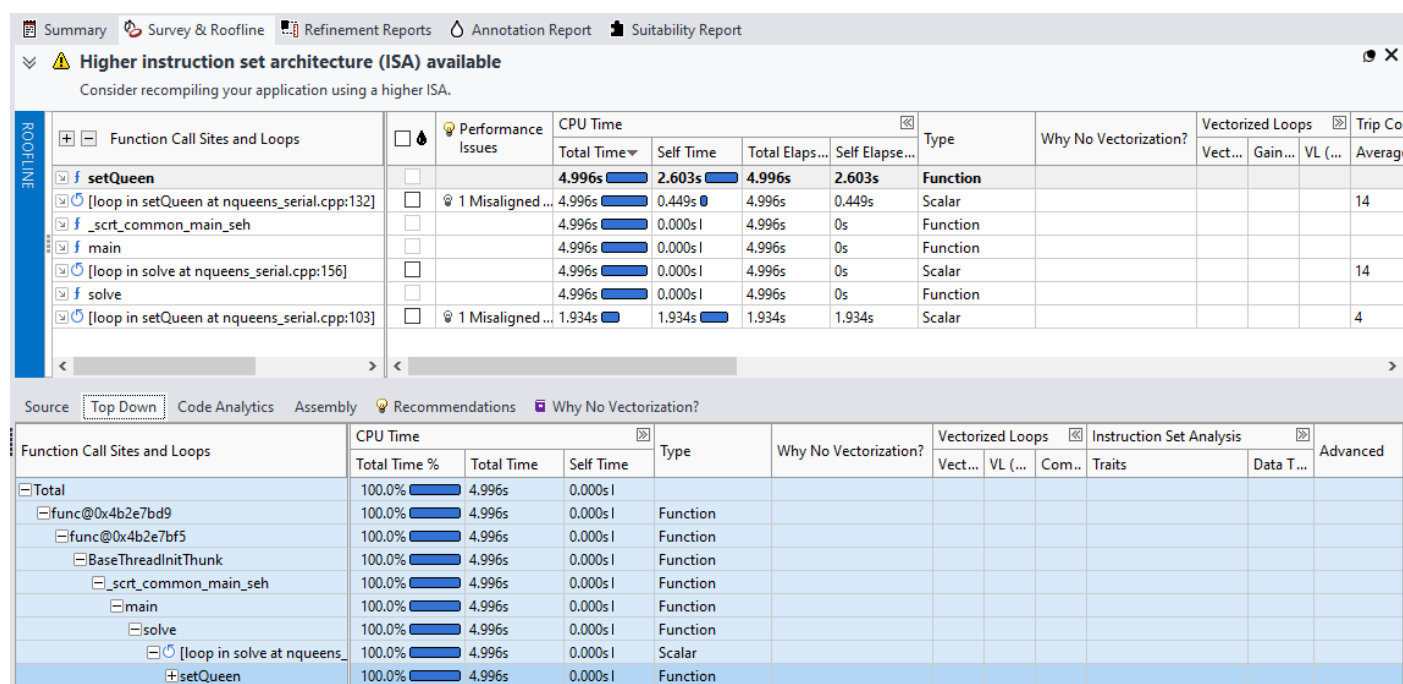
If you collect data using CLI, open the results in GUI using the following command:

```
advisor-gui ./1_nqueens_serial
```

If the result does not open automatically, click **Show Result**.

When you open the Vectorization and Code Insights result in GUI, Intel Advisor shows the **Summary** tab first. This window is a dashboard containing the main information about application execution, performance hints, and indication of vectorization problems in your application.

Switch to the **Survey & Roofline** to examine performance metrics for each loop/function and find the candidates for parallelization.



In the bottom pane of the **Survey & Roofline** report, click **Top Down** on the navigation toolbar to investigate functions/loops in hierarchy.

1. The **Total Time** column shows the time spent in a function or loop and all functions called from it. A row with a large **Total Time %** and multiple children with smaller total times are possible candidates for parallelism.
2. The **Self Time** column shows how much time was spent in each function or loop each time in was called. Loops or functions with significant self time values are possible candidates for distributing work.
3. The application spends the most time in the `setQueen()` function and calls itself recursively. This function is the parallelization candidate.

Mark Best Parallel Opportunities with Annotations

Annotations are subroutine calls or macro uses that you can use to mark places in serial parts of your program where Intel Advisor should assume your program's parallel execution and synchronization will occur. The annotations do not change the computations of your program, so your application runs normally.

1. Open the application source code `nqueens_serial.cpp` in your preferred editor.
2. Search for `ADVISOR_SUITABILITY_EDIT` and follow the directions in the sample code. Make four total edits to annotate the code:

- Uncomment `#include <advisor-annotate.h>`. This file is the include file that defines the annotations.
 - Uncomment `ANNOTATE_SITE_BEGIN(solve);`. This annotation marks the start of a parallel site that contains a single task in a loop.
 - Uncomment `ANNOTATE_ITERATION_TASK(setQueen);`. This annotation marks an iterative parallel task in a loop.
 - Uncomment `ANNOTATE_SITE_END();`. This annotation marks an end of a parallel site.
3. Save your edits and close the editor.
 4. [Rebuild](#) the target.

NOTE If the build fails due to the include file not found and undefined identifiers:

1. Go to **Project > 1_nqueens_serial Properties**.
 2. In the **C/C++ > Additional Include Directories**, change the Intel Advisor *year* version to the version installed on your machine. For example, `ADVISOR_2022_DIR`.
-

Model Threading Parallelism

Re-run the Threading perspective with additional analyses. Do *one* of the following:

Run Threading from GUI

1. In the **Analysis Workflow** pane, select the **Medium** accuracy level to configure the perspective automatically.

- 2.



Click the button to run the perspective.

At this accuracy level, Intel Advisor runs Survey, Characterization with trip counts, Suitability, and Dependencies analyses.

Important If you get the *Your configuration might be incomplete* message, click **Continue**. This warning message reminds you to make sure you have added annotations to your source code because Suitability and Dependencies analyses cannot run without them.

Run Threading from CLI on Linux OS

1. Run the Survey analysis to analyze performance.

```
advisor --collect=survey --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial
-- 1_nqueens_serial
```

2. Collect trip counts data.

```
advisor --collect=tripcounts --project-dir=./1_nqueens_serial --search-dir src:r=./
1_nqueens_serial -- 1_nqueens_serial
```

3. Model threading designs for the annotated functions/loops with the Suitability analysis.

```
advisor --collect=suitability --project-dir=./1_nqueens_serial --search-dir src:r=./
1_nqueens_serial -- 1_nqueens_serial
```

4. Identify data sharing problems that might prevent annotated functions/loops from parallelizing with the Dependencies analysis:

```
advisor --collect=dependencies --project-dir=./1_nqueens_serial --search-dir src:r=./
1_nqueens_serial -- 1_nqueens_serial
```

Run Threading from CLI on Windows OS

1. Run the Survey analysis to analyze performance.

```
advisor --collect=survey --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial  
-- 1_nqueens_serial.exe
```

2. Collect trip counts data.

```
advisor --collect=tripcounts --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial -- 1_nqueens_serial.exe
```

3. Model threading designs for the annotated functions/loops with the Suitability analysis.

```
advisor --collect=suitability --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial -- 1_nqueens_serial.exe
```

4. Identify data sharing problems that might prevent annotated functions/loops from parallelizing with the Dependencies analysis:

```
advisor --collect=dependencies --project-dir=./1_nqueens_serial --search-dir src:r=./1_nqueens_serial -- 1_nqueens_serial.exe
```

Examine the Results

If you collect data using GUI, Intel Advisor automatically opens the results when the collection completes.

If you collect data using CLI, open the results in GUI using the following command:

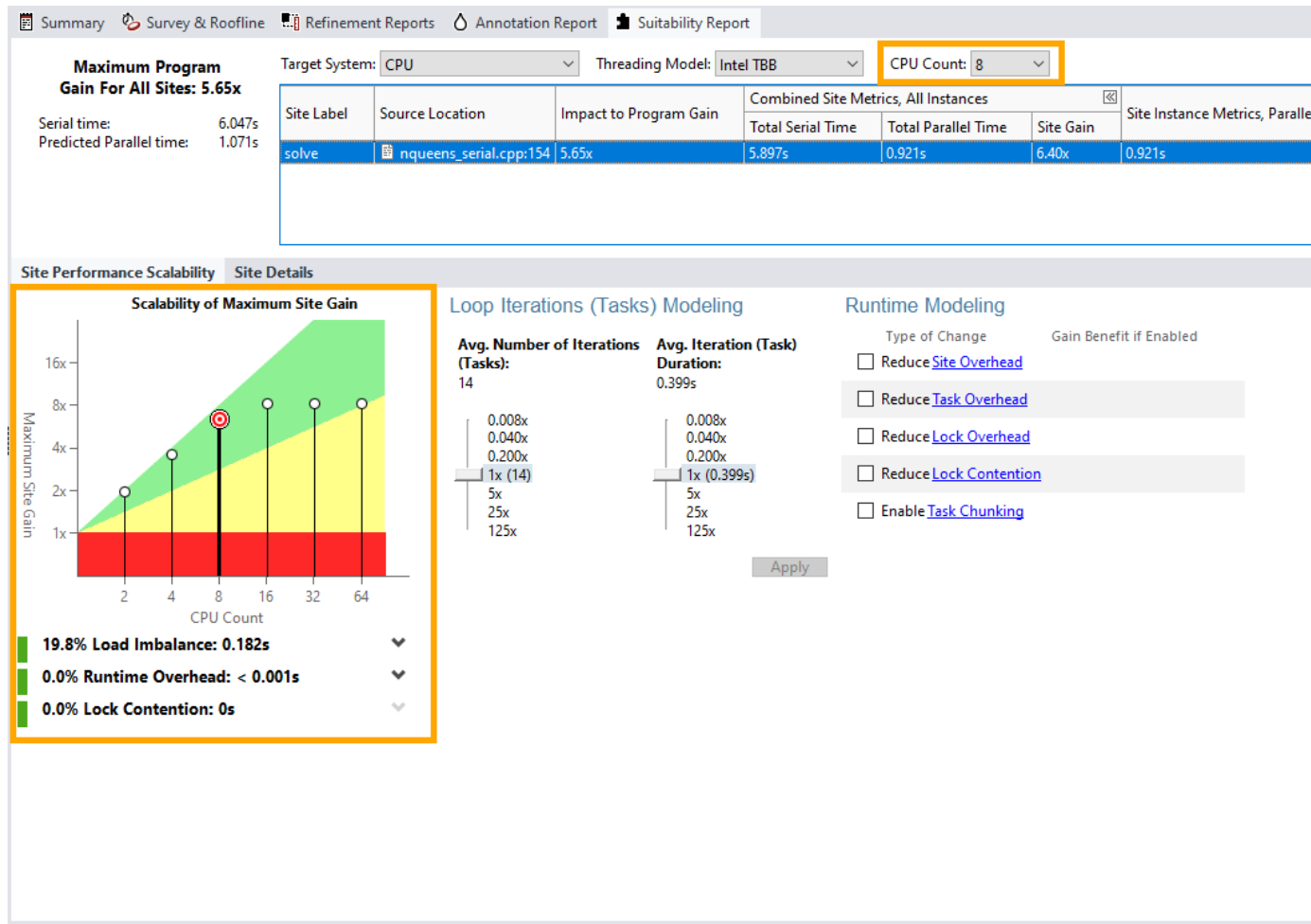
```
advisor-gui ./1_nqueens_serial
```

If the result does not open automatically, click **Show Result**.

When the Threading report opens, examine the application performance modeled with parallelism.

1. Go to the **Suitability** report tab to examine how parallelization can improve the performance:

- For the annotated loop at `nqueens_serial.cpp:154`, the Intel Advisor predicts the performance speedup around 1.80x for default configuration parameters.
- As the **Scalability of Maximum Site Gain** diagram shows, for CPU count from 2 to 16, the performance speedup increases. For the CPU count higher than 16, the performance speedup is the same, because the corresponding bull-eye dots are on the same line. Most of the dots on the diagram are located in the green zone, but from the 16 CPU, the higher the CPU count, the closer it is to the yellow zone. This means that the predicted speedup is worth an effort if you parallelize the loop for up to 16 CPUs. Parallelizing the loop to run on more than 16 CPUs might require more time and/or effort, but will result in the same speedup and might cause performance issues.



- Examine the three percentage metrics below the diagram. Notice that for the default CPU count of 8, the metrics are all green, which means that there are no performance issues. You are recommended to parallelize the loop for up to 8 CPUs to achieve optimal performance.
- Change the **CPU Count** to **16** to see the details about the predicted performance for this case. Notice that the corresponding dot is located closer to the yellow zone than the dots on the left from it. The **Load Imbalance** metric is yellow and is around 44%. The high load imbalance causes the predicted maximum speedup to be not enough to justify the effort needed to refactor your application. Consider investigating to understand how to optimize it.
- Experiment with the CPU count, threading model, and other parameters to see how they might affect the performance.
- Go to the **Refinement** reports tab to see if the annotated loops have dependencies that prevent parallelism.

The screenshot displays the Intel Advisor interface with the **Refinement Report** selected. The top pane shows the source code for `nqueens_serial.cpp` with annotations for dependencies: **RAW:1**, **WAR:1**, and **WAW**. The middle pane, **Problems and Messages**, lists several dependency issues, with **P3** (Read after write dependency) selected. The bottom pane, **Read after write dependency: Code Locations**, provides a detailed view of the race condition, showing that multiple tasks may increment `nrOfSolutions` simultaneously.

ID	Type	Site Name	Sources	Modules	State
P1	Parallel site information	solve	nqueens_serial.cpp	1_nqueens_serial.exe	✓ Not a problem
P3	Read after write dependency	solve	nqueens_serial.cpp	1_nqueens_serial.exe	New
P4	Write after write dependency	solve	nqueens_serial.cpp	1_nqueens_serial.exe	New
P5	Write after read dependency	solve	nqueens_serial.cpp	1_nqueens_serial.exe	New

ID	Instruction Address	Description	Source	Function	Variable references	Module	State
X3	0x4012a2	Read	nqueens_serial.cpp:126	setQueen	nrOfSolutions	1_nqueens_serial.exe	New
X4	0x4012a2	Write	nqueens_serial.cpp:126	setQueen	nrOfSolutions	1_nqueens_serial.exe	New
X5	0x40130d	Parallel site	nqueens_serial.cpp:154	solve		1_nqueens_serial.exe	New

- In the top pane of the **Refinement Report**, notice **RAW** (read after write), **WAR** (write after read), and **WAW** (write after write) dependencies in the loop in `solve` at `nqueens_serial.cpp:154`.
- From the top pane, select the loop in `solve` at `nqueens_serial.cpp:154`.
- In the **Problems and Messages** pane, examine the dependency problems found in the loop in more details. Select one of the problems to see more information. For example, select the **Read after write** dependency.
- In the **Code Locations** pane, examine the source of the Read after write dependency: The instructions reference the `nrOfSolutions` variable as the **Variable Reference** column shows. This means that a *race condition* happens because multiple tasks may try to increment the same variable at the same time.

You should fix the dependencies before applying threading to the application.

Next Steps

- Fix the dependencies found in the annotated loops. From the sample application source code, search for `ADVISOR CORRECTNESS EDIT` and follow the directions in the sample code to fix the problems (make six total edits).
- Rebuild the application and rerun the Threading perspective with the **Medium** accuracy (run the Survey, Trip Counts, Suitability, and Dependencies analyses).
- Make sure there are no dependencies found and your fixes did not negatively impact the predicted maximum speedup. Notice that the predicted speedup is higher and the load imbalance is green and does not impact the estimated performance anymore for the CPU count up to 8.

4. When you decide the predicted maximum speedup benefit is worth the effort to add parallelism to your target, replace annotations with parallel framework code.

This sample application already has the versions with replaced annotations with parallel framework code. Examine the following files:

Parallel Framework	File
Intel® Cilk™ Plus	3_nqueens_cilk.cpp
OpenMP*	3_nqueens_omp.cpp
Intel® Threading Building Blocks (Intel® TBB)	3_nqueens_tbb.cpp

5. Build the parallel version of the sample.
6. Test the resulting parallel application for correctness and verify its actual parallel performance using [other Intel Advisor perspectives](#), the Intel® Inspector, and Intel® VTune™ Profiler.

See Also

[User Guide: Threading Perspective](#)

[User Guide: Add Parallelism to Your Program](#)

Next Steps

- Learn more about Intel Advisor perspectives and other features in the [Intel Advisor User Guide](#).
- Explore other resources available for Intel Advisor perspectives:
 - [Roofline Resources for Intel® Advisor Users](#)
 - [Offload Modeling Resources for Intel® Advisor Users](#)
 - [Vectorization Resources for Intel® Advisor Users](#)
- Learn about other Intel Advisor capabilities and workflows using step-by-step instructions accompanied with sample applications in the [Intel Advisor Cookbook](#).
- Run Intel Advisor on your own code.

Tip

- Make sure you build an optimized binary of your application in release mode using settings designed to produce the most accurate and complete analysis results.
 - To minimize excessive execution time, try the techniques in in the [Intel Advisor User Guide: Minimize Analysis Overhead](#).
-

Learn More

Resource	Description
Intel Advisor User Guide	Refer to this guide for instructions to get started with the command line, detailed information on analysis types, information on how to use the GUI, and more.
Vectorization Resources for Intel® Advisor Users	View the most useful resources that can help you achieve better performance of your application using vectorization.
Roofline Resources for Intel® Advisor Users	View the most useful resources that can help you identify hardware-imposed ceilings using Intel Advisor CPU/GPU Roofline perspectives.

Resource	Description
Intel Advisor Cookbook	Explore typical use-cases of Intel Advisor. Follow the step-by-step instructions to help effectively use more cores, vectorization, or heterogeneous processing.
Flow Graph Analyzer User Guide	Explore a built-in graphical tool that helps you visualize and analyze graphs of a oneAPI Threading Building Blocks (oneTBB), OpenMP*, and SYCL applications.
Analyze Performance Remotely and Visualize Results on a Local macOS* System	View a step-by-step instruction how to visualize Intel Advisor perspective results on a macOS machine.
Intel Advisor Release Notes and New Features	Explore new features of Intel Advisor.
Vectorization Tutorial for Windows* OS	View tutorials that can help you experiment with Intel Advisor sample applications and run different perspectives.
Vectorization Tutorial for Linux* OS	
Threading Tutorial for Windows* OS	
Threading Tutorial for Linux* OS	
Roofline Tutorial for Windows* OS	
Offline Resources	One of the key Vectorization perspective features is GUI-embedded advice on how to fix vectorization issues specific to your code. To help you quickly locate information that augments that GUI-embedded advice, the Intel Advisor provides offline compiler <i>mini-guides</i>. You can also find offline Recommendations and Compiler Diagnostic Details advice libraries in the same location as the mini-guides. Each <i>issue</i> and <i>recommendation</i> in these HTML files is collapsible/expandable. Linux* OS: Available offline documentation is installed inside <code><advisor-install-dir>/documentation/<locale>/</code>. Windows* OS: Available offline documentation is installed inside <code><advisor-install-dir>\documentation\<locale>\</code>.

NOTE

You may encounter the following known issues when using the following to view documentation:

- Microsoft Windows Server* 2012 system: Trusted site prompt appears. Solution: Add *about:internet* to the list of trusted sites in the **Tools > Internet Options > Security** tab. You can remove after you finish viewing the documentation.
 - Microsoft Internet Explorer* 11 browser: Topics do not appear when you select them in the TOC pane. Solution: Add *http://localhost* to the list of trusted sites in the **Tools > Internet Options > Security** tab. You can remove after you finish viewing the documentation.
 - Microsoft Edge browser:
 - Context-sensitive (also known as F1) calls to a specific topic open the title page of the corresponding document instead. Solution: Use a different default browser.
 - Panes are truncated and a proper style sheet is not applied. Solution: Use a different default browser.
-

Notices and Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Microsoft, Windows, and the Windows logo are trademarks, or registered trademarks of Microsoft Corporation in the United States and/or other countries.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.